

Angular Learning Series

Topics 28 to 32

Angular Core — Components & Architecture

Generated: 17 May 2026

Topic 28 — Angular Template Syntax (Interpolation, Property Binding, Event Binding, Two-Way Binding)

Topic 29 — Built-in Control Flow (@if, @for, @switch)

Topic 30 — Built-in Directives (ngClass, ngStyle, ngTemplateOutlet)

Topic 31 — Content Projection (ng-content)

Topic 32 — ViewChild, ViewChildren, ContentChild, ContentChildren

TOPIC 28 — Angular Template Syntax

(Interpolation, Property Binding, Event Binding, Two-Way Binding)

Angular Core — Components & Architecture

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner-Intermediate — The individual syntaxes are simple, but knowing which binding to use, when, and why requires understanding Angular's data flow model, which trips up developers coming from plain HTML or other frameworks.

Angular Relevance: Critical — Template syntax is the bridge between your TypeScript component logic and the HTML the user sees. Every Angular component you write uses these four mechanisms; getting them wrong causes bugs that are hard to trace.

Section 1 — Explanation

THE DASHBOARD ANALOGY

Imagine a car dashboard. The speedometer shows you data from the engine — you can read it, but you can't change the engine speed by touching the gauge. That's interpolation and property binding — data flows one way, from the engine (component class) to the display (template).

The accelerator pedal does the opposite — you push it and it sends a signal back to the engine. That's event binding — user action in the template sends data back to the component class.

The cruise control dial does both at once — you set a speed, the car holds it, and the dial always reflects the current speed. That's two-way binding — data flows in both directions simultaneously.

TECHNICAL EXPLANATION

Angular templates are compiled HTML. The Angular compiler processes special syntax inside your HTML to wire up TypeScript logic to the DOM. There are four core binding mechanisms:

Mechanism	Syntax	Direction	Use Case
Interpolation	{{ expression }}	Class -> Template	Display text values
Property Binding	[property]="expression"	Class -> Template	Set DOM/component properties
Event Binding	(event)="handler()"	Template -> Class	Respond to user actions
Two-Way Binding	[(ngModel)]="property"	Both directions	Form inputs

Key insight: Square brackets [] mean "I'm reading from the class." Parentheses () mean "I'm sending to the class." Banana-in-a-box [()] is both combined.

THE FOUR MECHANISMS IN DETAIL

1 — Interpolation {{ }} Evaluates a TypeScript expression and converts the result to a string, then inserts it into the DOM as text content. It's syntactic sugar for a one-way string property binding.

```

{{ title }}          -> reads component.title, renders as text
{{ 2 + 2 }}          -> evaluates to "4"
{{ user?.name }}     -> safe navigation (optional chaining in templates)
{{ price | currency }} -> with a pipe

```

Important limit: Interpolation only produces text. You cannot use it to set DOM attributes that expect non-string values (like [disabled], [src], [class]).

2 — Property Binding [property]="expression" Sets a DOM element property or a child component's @Input() property to a TypeScript expression's value. This is evaluated as JavaScript, not a string.

```

<img [src]="imageUrl">          <!-- sets the .src property -->
<button [disabled]="isLoading"> <!-- sets the .disabled property (boolean) -->
<app-card [title]="pageTitle"> <!-- sets child @Input() -->

```

Attribute vs Property binding — this is a common confusion point. HTML attributes initialise the DOM; properties are the live JavaScript values. [attr.colspan]="n" is attribute binding; [value]="n" is property binding.

3 — Event Binding (event)="handler(\$event)" Listens for a DOM event on an element (or a custom event from a child component) and calls a method on the component class.

```

<button (click)="save()">Save</button>
<input (keyup)="onKey($event)">      <!-- $event = native DOM event -->
<app-form (submitted)="onSubmit($event)"> <!-- custom @Output() event -->

```

\$event is a special Angular template variable that carries the event payload — either a native DOM Event object or whatever type you defined for your @Output() EventEmitter.

4 — Two-Way Binding [(ngModel)]="property" Combines a property binding and an event binding in one shorthand. Under the hood, [(ngModel)] is exactly:

```

<input [ngModel]="username" (ngModelChange)="username = $event">

```

This pattern works with ngModel (from FormsModule) but also with any component that exposes a matching @Input() named X and an @Output() named XChange. That pattern is called the banana-in-a-box convention.

Section 2 — Connection to Prior Concepts

Direct dependencies: - Topic 4 — Arrow Functions & this Binding: event handlers in templates call class methods — understanding this prevents methods losing their context - Topic 10 — Destructuring & Spread: expressions inside {{ }} and [] bindings can use destructuring - Topic 19 — Optional Chaining: the ?. safe navigation operator works inside template expressions - Topic 23 — Angular Signals: signal() values are read in templates using signalName() — this is a special form of interpolation/property binding - Topic 24 — OnPush Change Detection: how Angular knows when to re-evaluate bindings is driven by change detection — these bindings are what gets checked - Topic 25 — Lifecycle Hooks: ngOnChanges fires when @Input() property-bound values change - Topic 26 — @Input() & @Output(): property binding targets @Input() properties; event binding listens to @Output() emitters - Topic 27 — Services & DI:

data displayed via interpolation typically comes from a service injected into the component

Bridging note: In Topic 26 you learned that `@Input()` and `@Output()` are the public API of a component. Template syntax is how you use that API from the outside. Property binding is how you pass data in; event binding is how you listen for data coming out. These two topics are two sides of the same coin.

Most directly extends: Topic 26 — `@Input()` & `@Output()`

Section 3 — Code Example

PLAIN JS/TS — THE CORE CONCEPT IN ISOLATION

```
// No Angular yet – just demonstrating what Angular is automating

class UserCard {
  name = 'Alice';
  isActive = true;

  greet() {
    console.log(`Hello, ${this.name}`);
  }
}

const card = new UserCard();

// Interpolation equivalent – manually reading a property and writing to DOM
document.getElementById('name').textContent = card.name;

// Property binding equivalent – setting a DOM property
const btn = document.getElementById('btn');
btn.disabled = !card.isActive;

// Event binding equivalent – attaching a listener
btn.addEventListener('click', () => card.greet());

// Angular automates ALL of the above and keeps it in sync reactively.
```

REAL ANGULAR EXAMPLE

```
@Component({
  selector: 'app-user-card',
  standalone: true,
  imports: [FormsModule, CommonModule],
  template: `
    <!-- INTERPOLATION -->
    <h2>{{ title }}</h2>
    <p>Items in cart: {{ cartItems.length }}</p>
    <p>Logged in as: {{ user?.displayName ?? 'Guest' }}</p>

    <!-- PROPERTY BINDING -->
    <img [src]="avatarUrl" [alt]="user?.displayName">
    <button [disabled]="isSubmitting">
      {{ isSubmitting ? 'Saving...' : 'Save' }}
    </button>
```

```

<app-status-badge [level]="currentStatus" [animate]="true">
</app-status-badge>
<td [attr.colspan]="columnSpan">Cell</td>
<div [class.active]="isActive">Profile</div>
<div [style.color]="isError ? 'red' : 'green'">Status</div>

<!-- EVENT BINDING -->
<button (click)="onSave()">Save</button>
<button (click)="onButtonClick($event)">Click Me</button>
<input (keyup.enter)="onEnterPressed()">
<input (keyup)="onKeyUp($event)">

<!-- TWO-WAY BINDING -->
<input [(ngModel)]="username" placeholder="Enter username">
<p>Hello, {{ username }}!</p>
<app-counter [(count)]="currentCount"></app-counter>
<p>Current count: {{ currentCount }}</p>
`
})
export class UserCardComponent {
  title = 'User Profile';
  cartItems = ['book', 'pen', 'lamp'];
  avatarUrl = 'https://example.com/avatar.jpg';
  isSubmitting = false;
  isActive = true;
  isError = false;
  currentStatus = 'online';
  columnSpan = 2;
  user: { displayName: string } | null = null;
  username = '';
  currentCount = 0;

  onSave(): void {
    this.isSubmitting = true;
    setTimeout(() => this.isSubmitting = false, 1500);
  }

  onButtonClick(event: MouseEvent): void {
    console.log('Clicked at:', event.clientX, event.clientY);
  }

  onKeyUp(event: KeyboardEvent): void {
    console.log('Key pressed:', event.key);
  }

  onEnterPressed(): void {
    console.log('Enter pressed, submitting...');
  }
}

```

BANANA-IN-A-BOX CONVENTION — CUSTOM TWO-WAY BINDING

```

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <button (click)="decrement()">-</button>
    <span>{{ count }}</span>
  `
})

```

```

    <button (click)="increment()">+</button>
  ,
})
export class CounterComponent {
  @Input() count = 0;
  @Output() countChange = new EventEmitter<number>();

  increment(): void {
    this.countChange.emit(this.count + 1);
  }

  decrement(): void {
    this.countChange.emit(this.count - 1);
  }
}
// Parent usage: <app-counter [(count)]=myCount"></app-counter>

```

Section 4 — Before & After Refactor

BEFORE — THE WRONG WAY

```

@Component({
  template: `
    <!-- MISTAKE 1: Using interpolation where property binding is needed -->
    <!-- Result: the button is never actually disabled -->
    <button disabled="{{ isLoading }}">Save</button>

    <!-- MISTAKE 2: String interpolation for src causes a broken image request -->
    

    <!-- MISTAKE 3: Attribute binding where property binding works fine -->
    <input [attr.value]="username">

    <!-- MISTAKE 4: $event here is a DOM Event object, not a string -->
    <input (change)="username = $event">
  ,
})

```

What breaks: - `disabled="{{ isLoading }}"` sets the attribute to the string "true" or "false". The presence of the disabled attribute regardless of value disables the button. So `disabled="false"` still disables it. - `src="{{ imageUrl }}"` causes a network request for a literal URL containing `{{ imageUrl }}` before Angular processes it. - `$event` on a (change) event is the DOM Event object — you want `$event.target.value` for the input's text value.

AFTER — THE CORRECT WAY

```

@Component({
  template: `
    <!-- Property binding passes a real boolean to the .disabled DOM property -->
    <button [disabled]="isLoading">Save</button>

    <!-- Property binding for src – no flicker, no broken request -->
    <img [src]="imageUrl">

    <!-- Two-way binding keeps username in sync automatically -->
  ,
})

```

```

<input [(ngModel)]="username">

<!-- When you need the raw $event, access .target.value explicitly -->
<input (change)="onInputChange($event)">
,
})
export class GoodFormComponent {
  isLoading = true;
  imageUrl = 'https://example.com/photo.jpg';
  username = '';

  onInputChange(event: Event): void {
    this.username = (event.target as HTMLInputElement).value;
  }
}

```

Why this matters in Angular: Change detection re-evaluates bindings on every cycle. Using the wrong binding type means Angular is setting the wrong thing (attributes vs properties), leading to bugs that appear to work in simple cases but break with dynamic data.

Section 5 — Common Mistakes & Misconceptions

MISTAKE 1 — Using Interpolation for Boolean HTML Properties

What the mistake looks like:

```

<button disabled="{{ isSubmitting }}">Save</button>
<input readonly="{{ isReadOnly }}">

```

Why developers make it: They see that `{{ }}` works for text content and assume it works everywhere.

How to avoid it: Ask: "Is this a boolean that controls behaviour?" If yes, use property binding `[disabled]="isSubmitting"`.

Angular-specific consequence: `disabled="{{ false }}"` renders as `disabled="false"` in the DOM, but the presence of the `disabled` attribute in HTML means "disabled" — the string value is ignored. Your button will be permanently disabled even when `isSubmitting` is false.

MISTAKE 2 — Confusing `$event` Type Across Different Event Bindings

What the mistake looks like:

```

// In template: <input (input)="onType($event)">
onType(event: string): void { // Wrong type
  console.log(event.toUpperCase()); // Runtime error
}

```

Why developers make it: With `ngModel`, `(ngModelChange)` gives you the raw value (a string). With native DOM events like `(input)`, `$event` is the full `InputEvent` object.

How to avoid it: - Native DOM events (click, input, keyup): `$event` is a DOM Event -> use `(event.target as HTMLInputElement).value` - `ngModelChange`: `$event` is the raw value (string, number, etc.) - Custom `@Output()` emitters: `$event` is whatever type you passed to `emit()`

Angular-specific consequence: TypeScript may not catch this at compile time if you type the parameter as any, leading to silent runtime errors.

MISTAKE 3 — Calling Functions With Side Effects in Interpolation

What the mistake looks like:

```
<!-- Called on EVERY change detection cycle -->
<p>{{ getUserFullName() }}</p>
<p>{{ calculateTotal() }}</p>
```

Why developers make it: It looks clean and simple.

How to avoid it: Use a component property (set once or in `ngOnInit`) instead of a method call. If the value must be computed, use Angular Signals' `computed()` (Topic 23) or a pure pipe (Topic 42).

Angular-specific consequence: With `OnPush` (Topic 24) this is somewhat mitigated, but in default change detection, a function called in a template can run hundreds of times per second during user interaction, causing severe performance degradation.

Section 6 — Do's and Don'ts

DO: - Use `[disabled]="booleanExpression"` for boolean DOM properties - Use `[src]`, `[href]`, `[routerLink]` for dynamic URLs - Use `[(ngModel)]` for simple form inputs in template-driven forms - Type `$event` explicitly in your event handler methods - Use `[attr.aria-label]`, `[attr.colspan]` for real HTML attributes that have no matching DOM property - Use `(keyup.enter)` shorthand for keyboard shortcuts

DON'T: - Use `disabled="{{ booleanExpression }}"` — attribute strings don't map to boolean properties - Use `src="{{ url }}"` — causes a failed network request before Angular resolves the expression - Use `[(ngModel)]` in reactive forms (Topic 33) — it conflicts with `FormControl` - Leave event handler parameters as any — you lose type safety and IDE help - Use `[attr.disabled]` when you mean `[disabled]` — they behave very differently - Write `(keyup)="if($event.key==='Enter') handler()"` in the template — Angular has built-in key filtering

Section 7 — Debugging Tips

BUG 1 — `[disabled]="false"` But Button Is Still Disabled

Symptom: Button appears greyed out and unclickable even though the condition is false in your component.

Cause: You used `disabled="{{ isDisabled }}"` instead of `[disabled]="isDisabled"`. The string "false" is a truthy value in HTML attribute binding.

Fix:

```
<button [disabled]="isDisabled">Save</button>
```


Angular-specific note: Look at the rendered HTML in DevTools. If you see `disabled="false"` in the DOM, that's the bug — it should be absent entirely.

BUG 2 — ERROR TypeError: Cannot read properties of undefined

Symptom: Console error on page load or after navigation, pointing to a template expression.

Cause: You're binding to a property on an object that's null or undefined when the component first renders.

Fix:

```
<!-- Option 1: Safe navigation operator -->
{{ user?.name }}

<!-- Option 2: With async pipe for Observables -->
{{ (user$ | async)?.name }}

<!-- Option 3: Default value with nullish coalescing -->
{{ user?.name ?? 'Loading...' }}
```

Angular-specific note: This is especially common when using OnPush change detection with async data.

BUG 3 — Two-Way Binding Not Working

Symptom: You type into an `<input [(ngModel)]="value">` field, but `this.value` in the component never updates.

Cause: `FormsModule` is not imported in the component's module or imports array.

Fix:

```
@Component({
  standalone: true,
  imports: [FormsModule], // Add this
  template: `<input [(ngModel)]="username">`
})
```

Angular-specific note: Angular won't throw a loud error for a missing `FormsModule` — the `[(ngModel)]` binding silently does nothing.

Section 8 — Real-World Applications

SCENARIO 1 — Search Bar With Live Filtering

```
@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [FormsModule, CommonModule],
  template: `
    <input
      [(ngModel)]="searchTerm"
      (ngModelChange)="onSearchChange()"
      placeholder="Search products..."
```

```

    >
    <button [disabled]="!searchTerm" (click)="clearSearch()">Clear</button>
    <p>Showing {{ filteredProducts.length }} of {{ products.length }} products</p>

    @for (product of filteredProducts; track product.id) {
      <app-product-card
        [product]="product"
        [isHighlighted]="product.name.toLowerCase().includes(searchTerm.toLowerCase())"
        (addToCart)="onAddToCart($event)"
      ></app-product-card>
    }
  }
})
export class ProductListComponent {
  searchTerm = '';
  products = [
    { id: 1, name: 'Angular Router', price: 0 },
    { id: 2, name: 'RxJS Essentials', price: 29 },
    { id: 3, name: 'TypeScript Handbook', price: 15 },
  ];
  filteredProducts = [...this.products];

  onSearchChange(): void {
    const term = this.searchTerm.toLowerCase();
    this.filteredProducts = this.products.filter(p =>
      p.name.toLowerCase().includes(term)
    );
  }

  clearSearch(): void {
    this.searchTerm = '';
    this.filteredProducts = [...this.products];
  }

  onAddToCart(productId: number): void {
    console.log('Adding product to cart:', productId);
  }
}

```

SCENARIO 2 — Dynamic Form Button State

```

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [FormsModule],
  template: `
    <form>
      <input [(ngModel)]="email" name="email" type="email"
        [class.error]="emailError" (blur)="validateEmail()">
      @if (emailError) {
        <span class="error-text">{{ emailError }}</span>
      }
      <input [(ngModel)]="password" name="password" type="password"
        (keyup.enter)="onLogin()">
      <button [disabled]="isLoading || !email || !password"
        [class.loading]="isLoading" (click)="onLogin()">
        {{ isLoading ? 'Signing in...' : 'Sign In' }}
      </button>
    </form>
  `
})

```

```

        @if (loginError) {
          <div class="error-banner" (click)="loginError = ''">
            {{ loginError }} x
          </div>
        }
      </form>
    },
  })
}

export class LoginComponent {
  email = '';
  password = '';
  emailError = '';
  loginError = '';
  isLoading = false;

  validateEmail(): void {
    this.emailError = this.email.includes('@') ? '' : 'Please enter a valid email';
  }

  onLogin(): void {
    if (!this.email || !this.password) return;
    this.isLoading = true;
    setTimeout(() => {
      this.isLoading = false;
      if (this.password !== 'correct') {
        this.loginError = 'Invalid credentials. Please try again.';
      }
    }, 1000);
  }
}

```

Section 9 — Practice Exercises

EXERCISE 1 — Beginner Build a standalone Angular component called GreetingCard. It should have a name property (defaulting to 'World') and display Hello, World! using interpolation. Add a text input using two-way binding so that as the user types a name, the greeting updates in real time. Add a button that resets the name back to 'World' using an event binding.

EXERCISE 2 — Intermediate Build a ToggleButton component that supports the banana-in-a-box two-way binding convention. The component should accept a `@Input()` active: boolean and emit a `@Output()` activeChange when toggled. The button should display "ON" or "OFF" based on the current state and apply a CSS class active using class binding. In a parent component, use `[(active)]="isFeatureEnabled"` and display the current state using interpolation.

EXERCISE 3 — Advanced Build a RatingWidget component that displays 5 star buttons. Use property binding to highlight stars up to the current rating (apply a filled CSS class), event binding to handle hover and click events (showing a preview rating on hover, setting the final rating on click), and export the selected rating to a parent component using two-way binding (`[(rating)]`). The parent should display the selected rating as text using interpolation and allow resetting via a button. Use `OnPush` change detection on the rating widget component.

Section 10 — Key Takeaways

Core Concepts: - Interpolation `{{ }}` — converts a TypeScript expression to a string and inserts it as DOM text content; read-only, class -> template - Property binding `[prop]` — sets a DOM element property or child component `@Input()` to a TypeScript expression's evaluated value; read-only, class -> template - Event binding (event) — listens for DOM or component events and calls a class method; write-only, template -> class - Two-way binding `[(ngModel)]` — shorthand combining `[ngModel]` and `(ngModelChange)`; bidirectional - `$event` — template-only keyword for the event payload; type depends on the event source - Banana-in-a-box `[(x)]` — works on any component with matching `@Input() x` and `@Output() xChange`

Quick Reference Table:

Syntax	Direction	Sets	Example
-----	-----	-----	-----
<code>{{ x }}</code>	-> DOM	Text content	<code>{{ user.name }}</code>
<code>[property]="x"</code>	-> DOM	DOM property	<code>[disabled]="loading"</code>
<code>[attr.x]="y"</code>	-> DOM	HTML attribute	<code>[attr.colspan]="n"</code>
<code>[class.x]="bool"</code>	-> DOM	Single CSS class	<code>[class.active]="isOn"</code>
<code>[style.x]="val"</code>	-> DOM	Inline style	<code>[style.color]='red'"</code>
<code>(event)="fn()"</code>	DOM ->	Class method	<code>(click)="save()"</code>
<code>[(ngModel)]="x"</code>	Both	Input + listener	<code>[(ngModel)]="name"</code>

Angular-Specific Rules: - Always import `FormsModule` when using `ngModel` - Never call methods with side effects in `{{ }}` — it runs every change detection cycle - Use `[disabled]`, not `disabled="{{ }}` for boolean DOM properties - Use `[src]`, `[href]` (property binding), not `src="{{ }}` to avoid broken resource requests - `$event` from native DOM events is a DOM Event object; `$event` from `@Output()` emitters is the emitted value's type

One-Line Memory Aid: Square brackets read from the class, parentheses write to the class, and banana-in-a-box does both — every Angular template is just these three ideas combined.

Section 11 — Thought-Provoking Question + Answer

Question: Two-way binding with `ngModel` seems perfect — it keeps things in sync automatically. So why does Angular's reactive forms system (Topic 33) actively avoid `ngModel` and use `FormControl` instead? Are they solving the same problem, or fundamentally different ones?

Answer:

`ngModel` two-way binding and `FormControl` look like they solve the same problem — keeping an input value in sync with a component property — but they operate at completely different layers.

Conceptual explanation: `ngModel` is template-driven. The source of truth lives in the template HTML, and Angular syncs it to the component class. `FormControl` is model-driven. The source of truth is a TypeScript object in the class, and the template just reflects it. This is the same distinction as "the dashboard controls the car" vs "the car controls the dashboard."

Why it matters: With `ngModel`, validation state, disabled state, and values are all scattered across your template. To validate, you add HTML attributes. To read the value programmatically,

you read `this.username`. To reset the form, you manually reset every bound property. With `FormControl`, a single object carries all of this: `control.value`, `control.valid`, `control.disable()`, `control.reset()`. You can test it without a template at all.

Angular-specific implication: At scale, template-driven forms become hard to test and maintain. A 10-field form with `ngModel` means 10 separate component properties plus HTML validation attributes scattered across the template. The same form with `FormGroup` is one structured TypeScript object you can inspect, serialise, and test independently of any component.

Practical rule: Use `ngModel` (two-way binding) for simple, isolated inputs where you just need a synced value and don't need validation or programmatic control. Use Reactive Forms (Topic 33) for any form where you need validation, async validators, conditional fields, programmatic resets, or unit tests.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts: - Getters and Setters — understanding JS property accessors helps you understand how Angular two-way binding conventions work under the hood

TypeScript Concepts: - Type Narrowing — important for correctly typing `$event` in event handlers
- Template Literal Types — relevant for understanding how Angular's template compiler does type-checking on binding expressions

Angular-Specific Concepts: - Topic 29 — Built-in Control Flow (`@if`, `@for`, `@switch`) — the next step: you can bind to properties, now learn how to show/hide and repeat elements - Topic 30 — Built-in Directives (`ngClass`, `ngStyle`) — extends `[class.x]` and `[style.x]` to handle multiple classes/styles at once - Topic 33 — Reactive Forms — where you graduate from `ngModel` to `FormControl` for complex forms - Topic 42 — Custom Pipes — pipes used inside `{{ }}` interpolation; understanding binding is a prerequisite - Topic 41 — Custom Directives — custom directives often receive `@Input()` via property binding; this topic is the foundation

Section 13 — Study Plan Checkpoint

This topic directly depends on: - Topic 4 — Arrow Functions (event handler this binding) - Topic 19 — Optional Chaining (safe navigation in templates) - Topic 23 — Angular Signals (signal values in templates use interpolation/property binding) - Topic 24 — OnPush Change Detection (bindings are what gets checked each cycle) - Topic 25 — Component Lifecycle Hooks (property binding triggers `ngOnChanges`) - Topic 26 — `@Input()` & `@Output()` (property binding targets `@Input()`; event binding listens to `@Output()`) - Topic 27 — Services & DI (data in bindings usually comes from injected services)

This topic unlocks: - Topic 29 — Built-in Control Flow — `@if` (condition) and `@for` rely on the same template expression syntax - Topic 30 — Built-in Directives — `[ngClass]` and `[ngStyle]` are just property bindings to Angular directives - Topic 31 — Content Projection — content projected with `ng-content` interacts with the parent template's bindings - Topic 33 — Reactive Forms — understanding why `ngModel` is insufficient is the prerequisite for Reactive Forms - Topic 41 —

Custom Directives — directives receive configuration via `@Input()` property binding

Production readiness assessment: You are ready to use template syntax confidently in production Angular apps — this is foundational material. The one area to practise more before production use is correctly typing `$event` in event handlers and understanding the attribute vs property distinction to avoid the disabled/src pitfalls.

Section 14 — Common Interview Questions

QUESTION 1 — Junior/Mid "What is the difference between property binding and attribute binding in Angular? When would you use `[attr.x]` instead of `[x]`?"

Strong answer covers: - HTML attributes set the initial DOM state; DOM properties reflect the current live state - Property binding (`[x]`) targets the JavaScript property on the DOM element object - Attribute binding (`[attr.x]`) sets the HTML attribute — needed when no matching DOM property exists (e.g. `colspan`, ARIA attributes like `aria-label`, SVG attributes) - `[disabled]` sets the boolean property; `[attr.disabled]` sets/removes the attribute string - Example: `<td [attr.colspan]="n">` because `colspan` is an attribute, not a standard DOM property

Weak answers miss: The distinction between the HTML attribute as initialiser vs the DOM property as live state.

QUESTION 2 — Mid-Senior "How does Angular's two-way binding actually work? Can you implement the banana-in-a-box pattern on a custom component?"

Strong answer covers: - `[(ngModel)]="x"` desugars to `[ngModel]="x" (ngModelChange)="x = $event"` - The pattern works on any component, not just inputs — via the `@Input() x` and `@Output() xChange` convention - The parent writes to the `@Input`, the child emits on `@Output`, and Angular updates the parent property - Code example showing both the child implementation and parent usage - Why this is preferable to passing a callback function (cleaner two-way contract)

Weak answers miss: That `[(x)]` is a shorthand that can work on any component — candidates often think it only works with `ngModel` on native inputs.

QUESTION 3 — Mid "Why should you avoid calling component methods inside template interpolation expressions? What's the Angular-specific risk?"

Strong answer covers: - Template bindings are re-evaluated every change detection cycle - In default change detection, a cycle runs on every browser event (click, keypress, timer, HTTP response, etc.) - A method call in `{{ getFullName() }}` runs potentially hundreds of times per second - Unlike a property, Angular cannot tell if the method's result has changed without calling it — so it always calls it - Solutions: use a component property, use `computed()` from Angular Signals, or use a pure pipe (which is memoised)

Weak answers miss: That the problem isn't just "inefficient" but that it can also cause infinite loops if the method has side effects that trigger further change detection.

Section 15 — Anti-Patterns to Avoid in Production

ANTI-PATTERN 1 — Expensive Methods in Template Expressions

```
<!-- Called on every single change detection cycle -->
<p>{{ getUserFullName() }}</p>
<p>{{ formatCurrency(product.price, 'GBP') }}</p>
<li *ngFor="let item of getFilteredItems()">{{ item.name }}</li>
```

Why it seems fine: It's readable, simple, and works perfectly in development with small datasets.

What breaks in production: With a list of 1,000 items, `getFilteredItems()` runs on every keypress, every mouseover, every HTTP response in the entire app. This causes UI stutters, dropped frames, and battery drain on mobile devices.

Correct pattern:

```
// Option 1: Compute once, store as a property
filteredItems = this.items.filter(...);

// Option 2: Angular Signals
filteredItems = computed(() => this.items().filter(...));
```

ANTI-PATTERN 2 — Direct DOM Manipulation Bypassing Angular's Binding System

```
export class ButtonComponent implements AfterViewInit {
  ngAfterViewInit() {
    const btn = document.querySelector('.submit-btn') as HTMLButtonElement;
    btn.disabled = true;           // Bypasses Angular
    btn.textContent = 'Submitting...'; // Bypasses Angular
    btn.style.opacity = '0.5';      // Bypasses Angular
  }
}
```

Why it seems fine: It achieves the visual result. Developers who come from jQuery or vanilla JS reach for this instinctively.

What breaks in production: Angular's change detection knows nothing about this change — the next cycle may overwrite your DOM modification. Breaks with SSR. Impossible to unit test.

Correct pattern:

```
export class ButtonComponent {
  isSubmitting = false;
  buttonText = 'Submit';
  // Template: <button [disabled]="isSubmitting">{{ buttonText }}</button>
}
```

ANTI-PATTERN 3 — Using String Interpolation for Security-Sensitive Bindings

```
<!-- Potentially allows XSS if url comes from user input -->
<a href="{{ userProvidedUrl }}">Click here</a>
```

Why it seems fine: Interpolation works for text content, so developers assume it works safely for attributes too.

What breaks in production: Angular's security model (built-in sanitisation) applies to property bindings — not string interpolation in attributes. A malicious user could inject `javascript:alert('XSS')` as the URL value.

Correct pattern:

```
<!-- Property binding – Angular sanitises href values automatically -->
<a [href]="userProvidedUrl">Click here</a>
```

Section 16 — Mental Model Summary

Core concept: Angular templates connect a TypeScript class to the DOM using four binding mechanisms — each handles a different direction and type of data flow.

Key rules: 1. `{{ }}` — text only, class -> DOM, converts to string 2. `[property]` — any JS value, class -> DOM, sets DOM properties (not attributes) 3. `(event)` — DOM -> class, calls a method, `$event` carries the payload 4. `[(ngModel)]` — bidirectional shorthand, requires `FormsModule` 5. `[attr.x]` — use for real HTML attributes that have no DOM property equivalent (ARIA, SVG, `colspan`) 6. Never call side-effectful methods in `{{ }}` — they run every change detection cycle

The single most important Angular rule: Use `[disabled]="boolExpr"` (property binding), never `disabled="{{ boolExpr }}"` (interpolation) — string "false" in an HTML attribute still means disabled.

Choosing the Right Binding:

You want to...	Use	Example
-----	-----	-----
Show text from the class	<code>{{ }}</code>	<code>{{ title }}</code>
Set a DOM property dynamically	<code>[prop]</code>	<code>[disabled]="loading"</code>
Set an HTML attribute	<code>[attr.x]</code>	<code>[attr.aria-label]="desc"</code>
Toggle a CSS class	<code>[class.x]</code>	<code>[class.active]="isOn"</code>
Set an inline style	<code>[style.x]</code>	<code>[style.color]="color"</code>
React to a user action	<code>(event)</code>	<code>(click)="save()"</code>
Sync an input field both ways	<code>[(ngModel)]</code>	<code>[(ngModel)]="email"</code>
Two-way bind a custom component	<code>[(x)]</code>	<code>[(count)]="myCount"</code>

TOPIC 29 — Built-in Control Flow

(@if, @for, @switch)

Angular Core — Components & Architecture

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner-Intermediate — The syntax itself is straightforward, but understanding when Angular re-renders, how track works in @for, and why the new block syntax replaced structural directives requires understanding change detection and DOM reconciliation.

Angular Relevance: Critical — Every Angular template that shows dynamic content uses control flow. You cannot build a real Angular app without @if and @for — they appear in virtually every component you will ever write.

Section 1 — Explanation

THE STAGE MANAGER ANALOGY

Imagine a theatre stage manager with three tools:

A spotlight — @if. The spotlight either illuminates a scene or it doesn't. When the condition is false, the scene doesn't just go dark — the actors leave the stage entirely. When the condition becomes true again, the actors walk back on and set up fresh. This is important: @if doesn't hide elements, it removes and recreates them in the DOM.

A photocopier — @for. You hand the photocopier one template (one actor's script) and a list of 50 names, and it stamps out 50 copies — one for each name. Change the list, and it figures out the minimum number of copies to add, remove, or move to match the new list. The track instruction tells the photocopier how to recognise which copy is which so it doesn't reprint everything from scratch.

A costume rack — @switch. The costume rack holds several outfits. Based on the character's role, exactly one outfit comes off the rack. Only one @case is ever active at a time.

TECHNICAL EXPLANATION

Angular 17 introduced a new built-in control flow syntax directly in the template compiler. It replaces the older structural directives (*ngIf, *ngFor, *ngSwitch) with a cleaner block-based syntax that looks similar to JavaScript's own if/for/switch.

Why the change? - The old *ngIf and *ngFor required importing CommonModule or NgIf/NgFor — the new syntax is built into the compiler, no imports needed - The new syntax supports type narrowing inside @if blocks (TypeScript knows the type is not null inside the block) - @for now requires track, making the performance contract explicit rather than optional - The block syntax is easier to read for developers who know JavaScript

Quick Comparison — Old vs New:

Old Syntax	New Syntax	Key Difference
-----	-----	-----
*ngIf="cond"	@if (cond) { }	Type-narrowing, no import r
*ngIf="cond; else tmpl"	@if (cond) { } @else { }	Inline else, no ng-template
*ngFor="let x of list; trackBy: fn"	@for (x of list; track x.id) { }	track is mandatory
[ngSwitch] + *ngSwitchCase	@switch (val) { @case (x) { } }	Single block

THE THREE BLOCKS IN DETAIL

@if — Conditional Rendering

```
@if (condition) {
  <!-- rendered when true -->
} @else if (otherCondition) {
  <!-- rendered when otherCondition true -->
} @else {
  <!-- rendered when all conditions false -->
}
```

Key behaviour: Angular removes the DOM nodes when the condition is false and creates them fresh when it becomes true. This means component lifecycle hooks run — `ngOnInit` fires every time the component enters the DOM, `ngOnDestroy` fires every time it leaves.

Type narrowing: Inside `@if (user) { }`, TypeScript automatically knows `user` is not null or undefined.

@for — List Rendering

```
@for (item of items; track item.id) {
  <!-- rendered once per item -->
} @empty {
  <!-- rendered when items is empty or null -->
}
```

`track` is mandatory. It tells Angular how to identify each item across re-renders. Angular uses the tracked value to perform the minimum DOM operations needed when the list changes.

Built-in loop variables — available inside `@for` without declaration:

Variable	Type	Meaning
-----	-----	-----
\$index	number	Current iteration index (0-based)
\$first	boolean	True for the first item
\$last	boolean	True for the last item
\$even	boolean	True for even-index items
\$odd	boolean	True for odd-index items
\$count	number	Total number of items in the list

@switch — Multi-Branch Rendering

```
@switch (expression) {
  @case (valueA) {
    <!-- rendered when expression === valueA -->
  }
  @case (valueB) {
    <!-- rendered when expression === valueB -->
  }
}
```

```

    }
    @default {
      <!-- rendered when no case matches -->
    }
  }
}

```

Uses strict equality (===). Only one @case renders at a time. Unlike JavaScript switch, there is no fallthrough — you don't need break.

Section 2 — Connection to Prior Concepts

Direct dependencies: - Topic 28 — Angular Template Syntax: @if, @for, and @switch use the same template expression syntax — property bindings, interpolation, and event bindings live inside the control flow blocks - Topic 24 — OnPush Change Detection: @if and @for interact directly with change detection — understanding when Angular re-evaluates these blocks requires knowing how change detection triggers - Topic 25 — Component Lifecycle Hooks: @if adding/removing a child component causes ngOnInit and ngOnDestroy to fire - Topic 23 — Angular Signals: @for re-renders when the signal array changes; @if re-evaluates when a signal value changes - Topic 9 — Array Methods: The arrays you pass to @for are often the result of .filter(), .map(), .sort()

Bridging note: In Topic 28 you learned that property binding and interpolation are how you connect TypeScript values to the DOM. Control flow is the layer above that — it determines whether and how many times those bindings are rendered at all. @if gates which bindings exist; @for multiplies them.

Most directly extends: Topic 28 — Angular Template Syntax

Section 3 — Code Example

PLAIN JS — THE CORE CONCEPT IN ISOLATION

```

const items = ['Apple', 'Banana', 'Cherry'];
const container = document.getElementById('list');

// @for equivalent – manually stamping out DOM nodes
items.forEach(item => {
  const li = document.createElement('li');
  li.textContent = item;
  container.appendChild(li);
});

// Problem: if items changes, how do you update the DOM efficiently?
// Angular's @for + track handles this diffing automatically.

// @if equivalent
const isLoggedIn = true;
if (isLoggedIn) {
  document.getElementById('welcome').style.display = 'block';
  // But note: this just hides it. Angular @if actually removes the DOM node.
}

```

REAL ANGULAR EXAMPLE

```
@Component({
  selector: 'app-dashboard',
  standalone: true,
  template: `
    @if (isLoading()) {
      <div class="spinner">Loading...</div>

    } @else if (error()) {
      <div class="error-banner">
        Error: {{ error() }}
        <button (click)="retry()">Retry</button>
      </div>

    } @else if (users().length === 0) {
      <p class="empty-state">No users found.</p>

    } @else {
      <h2>Users ({{ users().length }})</h2>
      <ul>
        @for (user of users(); track user.id) {
          <li
            [class.inactive]="!user.isActive"
            [class.first-item]="$first"
            [class.last-item]="$last"
          >
            <span class="index">{{ $index + 1 }} / {{ $count }}</span>
            <strong>{{ user.name }}</strong>

            @switch (user.role) {
              @case ('admin') {
                <span class="badge badge-red">Admin</span>
              }
              @case ('editor') {
                <span class="badge badge-blue">Editor</span>
              }
              @case ('viewer') {
                <span class="badge badge-grey">Viewer</span>
              }
              @default {
                <span class="badge">Unknown</span>
              }
            }

            @if (user.isActive) {
              <span class="status active">Active</span>
            } @else {
              <span class="status inactive">Inactive</span>
            }
          </li>

        } @empty {
          <li class="no-results">No users match the current filter.</li>
        }
      </ul>
    }
  `
})
```

```

    })
    export class DashboardComponent {
        isLoading = signal(true);
        error = signal<string | null>(null);
        users = signal<User[]>([]);

        constructor() {
            setTimeout(() => {
                this.isLoading.set(false);
                this.users.set([
                    { id: 1, name: 'Alice Chen', role: 'admin', isActive: true },
                    { id: 2, name: 'Bob Smith', role: 'editor', isActive: true },
                    { id: 3, name: 'Carol White', role: 'viewer', isActive: false },
                ]);
            }, 1000);
        }

        retry(): void {
            this.isLoading.set(true);
            this.error.set(null);
        }
    }
}

```

TRACK IN DEPTH — WHY IT MATTERS

```

// track $index – tells Angular items are identified by position
// If you insert an item at the start, Angular thinks ALL items changed
@for (item of items; track $index) {
    <app-item [data]="item"></app-item>
}

// track item.id – each item has a stable identity
// Insert at start: only one new DOM node created, rest are moved
@for (item of items; track item.id) {
    <app-item [data]="item"></app-item>
}

// For primitive arrays (strings, numbers) – track the value itself
@for (name of names; track name) {
    <li>{{ name }}</li>
}

```

Section 4 — Before & After Refactor

BEFORE — OLD STRUCTURAL DIRECTIVE SYNTAX

```

import { CommonModule } from '@angular/common'; // Required import

@Component({
    standalone: true,
    imports: [CommonModule], // Must import for *ngIf/*ngFor/*ngSwitch
    template: `
        <!-- Requires ng-template for else branch – verbose -->
        <div *ngIf="isLoading; else content">Loading...</div>
        <ng-template #content>

```

```

        <div *ngFor="let user of users; trackBy: trackById; let i = index">
            {{ i }}: {{ user.name }}
        </div>
    </ng-template>

    <!-- ngSwitch requires three separate directives -->
    <div [ngSwitch]="user.role">
        <span *ngSwitchCase="'admin'">Admin</span>
        <span *ngSwitchCase="'editor'">Editor</span>
        <span *ngSwitchDefault>Viewer</span>
    </div>
    ,
})
export class OldComponent {
    isLoading = true;
    users: User[] = [];

    // trackBy required as a method – not co-located with the template
    trackById(index: number, user: User): number {
        return user.id;
    }
}

```

What's wrong with this: - Requires importing CommonModule (or individual NgIf, NgFor directives) — extra boilerplate - The else branch requires a separate `<ng-template #ref>` — not visually adjacent - trackBy is a class method disconnected from the `@for` expression — easy to forget - `*ngSwitch` splits the switch logic across a wrapper element and child elements

AFTER — NEW BUILT-IN CONTROL FLOW

```

@Component({
    standalone: true,
    imports: [], // Nothing needed for control flow – it's built in
    template: `
        @if (isLoading) {
            <div>Loading...</div>
        } @else {
            @for (user of users; track user.id) {
                {{ user.name }}
            } @empty {
                No users.
            }
        }

        @switch (user.role) {
            @case ('admin') { <span>Admin</span> }
            @case ('editor') { <span>Editor</span> }
            @default { <span>Viewer</span> }
        }
    `,
})
export class NewComponent {
    isLoading = true;
    users: User[] = [];
    // No trackBy method needed – track expression is inline
}

```

Why this matters in Angular: Fewer imports, better TypeScript integration, mandatory track prevents the most common `@for` performance mistake, and `@else` branches are co-located — easier to read and maintain at scale.

Section 5 — Common Mistakes & Misconceptions

MISTAKE 1 — Using track \$index as the Default

What the mistake looks like:

```
@for (item of items; track $index) {  
  <app-card [data]="item"></app-card>  
}
```

Why developers make it: Angular requires track and `$index` is always available — so developers use it as the path of least resistance.

How to avoid it: Always ask: "Does this item have a stable unique identifier?" If yes (`item.id`, `item.uuid`, `item.slug`), use that. Use track `$index` only for static lists that never change order.

Angular-specific consequence: When the list reorders or an item is inserted anywhere except the end, Angular treats every item at a changed position as a different item entirely. Every `app-card` at those positions is destroyed and recreated — firing `ngOnInit` again, losing any internal component state.

MISTAKE 2 — Expecting @if to Hide Elements (Like `display: none`)

What the mistake looks like:

```
@if (showPanel) {  
  <!-- Developer expects panel to remember its state when showPanel flips back -->  
  <app-side-panel></app-side-panel>  
}
```

Why developers make it: CSS developers default to thinking about `display: none`. Both preserve the element and its state.

How to avoid it: Understand the distinction: `@if (false)` removes the DOM node and destroys the component. `@if (true)` creates a new instance. If you need to preserve component state, use `[hidden]` or `display: none` via a CSS class binding instead.

```
<!-- Hides without destroying – component instance persists -->  
<app-side-panel [hidden]="!showPanel"></app-side-panel>
```

Angular-specific consequence: `ngOnInit` fires every time `@if` becomes true — if `ngOnInit` triggers an HTTP call, you'll make a new API call every time the panel is shown.

MISTAKE 3 — Forgetting That @if Removal Triggers Child `ngOnDestroy`

What the mistake looks like:

```
@Component({...})  
export class TimerComponent implements OnInit {  
  ngOnInit() {
```

```

    // No unsubscribe – memory leak every time the component mounts
    interval(1000).subscribe(n => console.log(n));
  }
}
// Parent: @if (showTimer) { <app-timer></app-timer> }

```

Why developers make it: The leak only happens when showTimer toggles multiple times.

How to avoid it: Treat every component wrapped in @if as a component that may be created and destroyed many times. Always use takeUntilDestroyed() (Topic 21) or DestroyRef for subscriptions.

Angular-specific consequence: Each time @if shows the component, a new subscription is added. Memory usage grows linearly with each toggle, and multiple simultaneous subscriptions fire duplicate side effects.

Section 6 — Do's and Don'ts

DO: - Use track item.id (stable unique identity) in @for - Use @empty inside @for for clean empty-state handling - Use @if type narrowing — no ! assertions needed inside @if (obj) blocks - Use [hidden]="condition" when you need to preserve component state across toggles - Nest @if, @for, @switch freely — the block syntax is designed for composition - Keep @switch for role/status/type enum branching where cases are distinct values

DON'T: - Use track \$index as a default for lists that change at runtime - Wrap @for in an @if (items.length > 0) just to handle the empty case — use @empty - Assert non-null with obj!.property inside an @if (obj) block — it's redundant - Use @if when you actually want to hide-but-preserve — they are not the same - Reach for old *ngIf/*ngFor in new Angular 17+ code — use the new syntax consistently - Use @switch for range comparisons (> 10) — it uses ===, not range logic

Section 7 — Debugging Tips

BUG 1 — Component Re-initialises Every Time a Condition Toggles

Symptom: ngOnInit fires multiple times for a child component. API calls trigger repeatedly.

Cause: The component is wrapped in @if, and the condition is toggling. Every false -> true transition destroys the old instance and creates a new one.

Fix:

```

<!-- Option 1: Use [hidden] to preserve the instance -->
<app-expensive-panel [hidden]="!isVisible"></app-expensive-panel>

<!-- Option 2: Lift state out of the component into a parent signal or service -->

```

Angular-specific note: This is especially important for form components inside @if. A form inside @if resets entirely when the condition becomes false then true again.

BUG 2 — @for List Not Updating When Array Is Mutated

Symptom: You push a new item into an array (`this.items.push(newItem)`) but the `@for` list in the template doesn't update.

Cause: Angular's change detection compares object references, not contents. If you mutate the same array reference (push, splice, sort in place), Angular doesn't detect a change.

Fix:

```
// Mutation – Angular doesn't see a change
this.items.push(newItem);

// Replace the reference – Angular sees a new array reference
this.items = [...this.items, newItem];

// With Signals – mutation detection is automatic
this.items.update(current => [...current, newItem]);
```

Angular-specific note: This bug is most common with OnPush components (Topic 24).

BUG 3 — @switch @default Never Renders, Or Wrong @case Renders

Symptom: The switch expression has a value, but either the wrong case renders or `@default` shows when it shouldn't.

Cause: `@switch` uses strict equality (`===`). Type mismatches between the expression and the case value cause cases to never match.

Fix:

```
// Role is a number, but cases are strings – strict equality fails
// user.roleId = 1 (number)
@switch (user.roleId) {
  @case ('1') { <span>Admin</span> } // '1' !== 1 – never matches
}

// Ensure types match
@switch (user.roleId) {
  @case (1) { <span>Admin</span> } // 1 === 1 – matches
}
```

Angular-specific note: TypeScript will often catch this at compile time if the switch expression and case values have explicit types.

Section 8 — Real-World Applications

SCENARIO 1 — Notifications Feed With Status Badges

```
@Component({
  selector: 'app-notifications',
  standalone: true,
  template: `
    <div class="notifications-panel">
      <h3>Notifications ({{ unreadCount() }})</h3>
```

```

    @if (isLoading()) {
      <div class="skeleton-loader">
        @for (n of [1,2,3]; track n) {
          <div class="skeleton-item"></div>
        }
      </div>

    } @else {

      @for (notification of notifications(); track notification.id) {
        <div
          class="notification-item"
          [class.unread]="!notification.isRead"
          (click)="markAsRead(notification.id)"
        >
          @switch (notification.type) {
            @case ('message') { <span class="icon">Message</span> }
            @case ('alert') { <span class="icon">Alert</span> }
            @case ('success') { <span class="icon">Success</span> }
            @case ('warning') { <span class="icon">Warning</span> }
          }

          <div class="content">
            <strong>{{ notification.title }}</strong>
            <p>{{ notification.body }}</p>
            @if ($last) {
              <button (click)="loadMore()">Load more</button>
            }
          </div>

          @if (!notification.isRead) {
            <span class="unread-dot"></span>
          }
        </div>

      } @empty {
        <div class="empty-state">
          <p>You're all caught up!</p>
        </div>
      }
    }
  </div>
}

export class NotificationsComponent {
  isLoading = signal(false);
  notifications = signal<Notification[]>([]);
  unreadCount = computed(() => this.notifications().filter(n => !n.isRead).length);

  markAsRead(id: string): void {
    this.notifications.update(list =>
      list.map(n => n.id === id ? { ...n, isRead: true } : n)
    );
  }
}

```

SCENARIO 2 — Permission-Gated UI

```

@Component({
  selector: 'app-settings',
  standalone: true,
  template: `
    <h1>Settings</h1>

    @if (currentUser()) {
      @switch (currentUser()!.role) {

        @case ('admin') {
          <section><h2>System Configuration</h2>
            <app-system-settings></app-system-settings>
          </section>
          <section><h2>User Management</h2>
            <app-user-management></app-user-management>
          </section>
        }

        @case ('manager') {
          <section><h2>Team Settings</h2>
            <app-team-settings></app-team-settings>
          </section>
        }

        @default {
          <section><h2>My Preferences</h2>
            <app-personal-settings></app-personal-settings>
          </section>
        }
      }

      @if (currentUser()!.role !== 'member') {
        <section><h2>Billing</h2><app-billing></app-billing></section>
      }

    } @else {
      <app-loading-state message="Loading your settings..."></app-loading-state>
    }
  `
})
export class SettingsComponent {
  currentUser = signal<{ role: Role; name: string } | null>(null);
}

```

Section 9 — Practice Exercises

EXERCISE 1 — Beginner Build a standalone component called `TodoList`. It should have a `todos` array of objects with `id`, `text`, and `completed` properties. Use `@for` with proper track to render each todo. Use `@if` to show a checkmark next to completed items and an `@empty` block that displays "No todos yet!" when the array is empty. Add a button to add a new todo and verify the list updates correctly.

EXERCISE 2 — Intermediate Build a `StatusCard` component that accepts a status input of type `'loading' | 'success' | 'error' | 'empty'`. Use `@switch` to render a completely different layout for

each status — loading shows a spinner, success shows a green checkmark with a message, error shows a red warning with a retry button, and empty shows an illustration and call-to-action text. In a parent component, cycle through all four statuses using a button to verify each branch renders correctly.

EXERCISE 3 — Advanced Build a `DataTable` component that renders a list of employee records (`id`, `name`, `department`, `salary`, `isActive`). Use `@for` with `track employee.id` to render rows. Use `@if/@else` to show active vs inactive styling. Use `$even/$odd` to apply alternating row colours. Add a `filterDepartment` signal — when set, the table should only show employees from that department using a `computed()` signal for the filtered list. Add a sort button that reverses the list and verify that `track employee.id` correctly moves DOM nodes rather than recreating them.

Section 10 — Key Takeaways

Core Concepts: - `@if` — conditionally creates or destroys DOM nodes; not the same as CSS `hide/show`; triggers full component lifecycle - `@for` — renders a template once per item in a collection; requires `track` to identify items for efficient DOM reconciliation - `track` — tells Angular how to identify items across re-renders; stable unique IDs (`item.id`) are almost always correct; `$index` is a performance trap for dynamic lists - `@empty` — a companion block to `@for` that renders when the collection is empty - `@switch` — multi-branch rendering with strict equality (`===`); no fallthrough; `@default` is the catch-all - Built-in variables in `@for`: `$index`, `$first`, `$last`, `$even`, `$odd`, `$count` — no declaration needed - No imports needed — unlike `*ngIf/*ngFor`, the new syntax requires nothing in the imports array

Quick Reference Table:

Block	Syntax	DOM Behaviour	Key Option
-----	-----	-----	-----
<code>@if</code>	<code>@if (expr) { } @else { }</code>	Creates / destroys	<code>@else if</code> for chaining
<code>@for</code>	<code>@for (x of list; track x.id) { }</code>	Stamps per item	<code>@empty</code> for empty state
<code>@switch</code>	<code>@switch (val) { @case (x) { } }</code>	One branch active	<code>@default</code> for fallback

Angular-Specific Rules: - `@if` destroys components — `ngOnInit` re-fires on every `false -> true` transition - Always `track` by a stable unique property, not `$index`, for lists that change at runtime - `@if` type-narrows TypeScript — no `!` assertions needed inside the block - Do not mutate arrays in place with `push/splice` — replace the reference or use Signals - `@switch` uses `===` — ensure expression and case value types match exactly

One-Line Memory Aid: `@if` creates and destroys, `@for` stamps and tracks, `@switch` picks exactly one — together they control what exists, how many times, and which version.

Section 11 — Thought-Provoking Question + Answer

Question: `@if (condition)` removes the DOM node entirely when false, while `[hidden]="condition"` keeps it in the DOM but invisible. They both "hide" content from the user — so which one should you use, and does the choice actually matter at scale?

Answer:

The choice matters significantly at scale, and the right answer depends on what you're optimising for.

Conceptual explanation: `@if` and `[hidden]` represent two different philosophies: existence vs visibility. `@if` says "this content doesn't exist right now." `[hidden]` says "this content exists but isn't visible right now."

What this means in practice: - A component inside `@if` (false) has no JavaScript instance, no subscriptions, no memory footprint, no change detection cost - A component inside `[hidden]="true"` has a full live JavaScript instance — its `ngOnInit` has already run, its subscriptions are active, Angular's change detection still evaluates its bindings every cycle - A complex component tree that is `[hidden]` is still fully rendered in memory and checked on every change detection cycle

The Angular-specific implication: - Use `@if` (default choice) when the hidden content is expensive to maintain, has side effects (API calls in `ngOnInit`), or is unlikely to be shown - Use `[hidden]` only when you specifically need to preserve component state across visibility toggles and the component is cheap to keep alive

Practical rule: Start with `@if`. Switch to `[hidden]` only when you've identified a specific state preservation need that can't be solved by lifting state up into a signal or service.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts: - Array destructuring in loops — `@for` (`let {id, name} of users; track id`) uses destructuring directly in the loop expression

TypeScript Concepts: - Type narrowing and control flow analysis — TypeScript's own narrowing mirrors what `@if` does in templates - Discriminated unions — a natural fit for `@switch` — when your objects have a type field that narrows which properties exist

Angular-Specific Concepts: - Topic 30 — Built-in Directives (`ngClass`, `ngStyle`, `ngTemplateOutlet`) — the next layer: control their dynamic styling - Topic 31 — Content Projection (`ng-content`) — `@if` and `@for` inside projected content has specific behaviour - Topic 42 — Custom Pipes — pipes used inside `@for` expressions to transform displayed values - Topic 45 — Performance Optimisation (deferrable views) — `@defer` is the control flow block for lazy loading

Section 13 — Study Plan Checkpoint

This topic directly depends on: - Topic 9 — Array Methods (arrays passed to `@for` are often `.filter()/.map()` results) - Topic 23 — Angular Signals (signals as the data source for `@if` and `@for`) - Topic 24 — OnPush Change Detection (control flow blocks and when they re-evaluate) - Topic 25 — Component Lifecycle Hooks (`@if` triggers `ngOnInit/ngOnDestroy`) - Topic 28 — Angular Template Syntax (bindings live inside control flow blocks)

This topic unlocks: - Topic 30 — Built-in Directives — directives like `ngClass` and `ngStyle` are used inside `@for` and `@if` blocks constantly - Topic 31 — Content Projection — projected content often contains `@if/@for` blocks - Topic 33 — Reactive Forms — form arrays and conditional form fields use `@for` and `@if` - Topic 45 — Performance Optimisation — `trackBy` patterns, `@defer`, and pure pipes in loops all build on this foundation

Production readiness assessment: You are ready to use `@if`, `@for`, and `@switch` confidently in production — but commit to always using `track item.id` (not `$index`) and understanding the `@if-destroys-vs-hidden-hides` distinction before shipping components where state preservation matters.

Section 14 — Common Interview Questions

QUESTION 1 — Mid "What does the `track` expression do in Angular's `@for` block, and what happens if you always use `track $index`?"

Strong answer covers: - `track` gives Angular a way to identify each list item across re-renders by a stable key - Angular uses it to perform DOM reconciliation — move/update existing nodes rather than destroy and recreate - `track $index` tells Angular items are identified by position — insert at start causes all subsequent items to be treated as "changed" - Consequence: child component state lost, `ngOnInit` re-fires, animations reset, performance degrades - Correct approach: `track item.id` or another stable unique property

Weak answers miss: That `track $index` isn't always wrong — for static, append-only lists it's acceptable.

QUESTION 2 — Mid "What is the difference between using `@if` and `[hidden]` to conditionally show content in Angular? When would you choose one over the other?"

Strong answer covers: - `@if (false)` removes the DOM node entirely; destroys child component instances; `ngOnDestroy` fires - `[hidden]` applies `display: none`; component instance stays alive; subscriptions, change detection, memory all still active - `@if` is more efficient when content is rarely shown or has expensive `ngOnInit` side effects - `[hidden]` preserves component state across toggles — better for UI panels the user switches between frequently - Default choice should be `@if`

Weak answers miss: That `[hidden]` is not inherently "better" for performance — it keeps everything alive and still gets change detection cycles.

QUESTION 3 — Mid-Senior "How does the new Angular 17 built-in control flow (`@if`, `@for`) differ from the old structural directives (`*ngIf`, `*ngFor`)? Why was the change made?"

Strong answer covers: - New syntax is compiled directly by the Angular template compiler — no `CommonModule` or `NgIf/NgFor` imports needed - `@if` provides TypeScript type narrowing inside the block — old `*ngIf` did not - `@for` makes `track` mandatory — old `trackBy` was optional and often forgotten - `@empty` is a clean companion to `@for` — old approach required a separate `*ngIf` and `ng-template` - The block syntax is more readable and closer to JavaScript's native control flow syntax

Weak answers miss: The TypeScript type narrowing improvement.

Section 15 — Anti-Patterns to Avoid in Production

ANTI-PATTERN 1 — Nested @for Without Proper Tracking on Both Levels

```
@for (department of departments; track department.id) {  
  <h3>{{ department.name }}</h3>  
  @for (employee of department.employees; track $index) {  
    <!-- Inner loop uses $index – reorder employees and everything re-renders -->  
    <app-employee-row [employee]="employee"></app-employee-row>  
  }  
}
```

Why it seems fine: The outer loop is correctly tracked. The inner loop's performance issue is hidden.

What breaks in production: With 50 departments x 20 employees = 1,000 app-employee-row components, sorting employees by salary re-renders all 1,000 rows instead of moving existing DOM nodes.

Correct pattern:

```
@for (employee of department.employees; track employee.id) {  
  <app-employee-row [employee]="employee"></app-employee-row>  
}
```

ANTI-PATTERN 2 — Complex Logic Directly in @if Conditions

```
@if (user && user.roles && user.roles.includes('admin') && !user.isSuspended && featureFlags.get  
  <app-admin-dashboard></app-admin-dashboard>  
)
```

Why it seems fine: It works correctly and is self-contained in the template.

What breaks in production: This condition is re-evaluated on every change detection cycle. Hard to unit test. Violates separation of concerns.

Correct pattern:

```
// Computed property in the class  
get canAccessAdminDashboard(): boolean {  
  return !!this.user &&  
    this.user.roles?.includes('admin') === true &&  
    !this.user.isSuspended &&  
    this.featureFlags.get('newDashboard') === true;  
}  
// Template: @if (canAccessAdminDashboard) { ... }
```

ANTI-PATTERN 3 — Using @for Over Untracked Computed Arrays

```
@Component({  
  template: `  
    @for (item of getVisibleItems(); track $index) {  
      <app-item [data]="item"></app-item>  
    }  
  `
```

```

    }
  },
  })
export class BadListComponent {
  private allItems = signal<Item[]>([]);
  // Called on every change detection cycle – creates new array reference each time
  getVisibleItems(): Item[] {
    return this.allItems().filter(i => i.visible);
  }
}

```

Why it seems fine: Reads cleanly and the filtering logic is centralised in the method.

What breaks in production: The method runs on every change detection cycle and returns a new array reference every time. With `track $index`, Angular sees a new list every cycle and re-renders all items.

Correct pattern:

```

export class GoodListComponent {
  private allItems = signal<Item[]>([]);
  // computed() is memoised – only recalculates when allItems() changes
  visibleItems = computed(() => this.allItems().filter(i => i.visible));
}
// Template: @for (item of visibleItems(); track item.id) { ... }

```

Section 16 — Mental Model Summary

Core concept: `@if`, `@for`, and `@switch` are Angular's built-in template compiler blocks that control whether DOM nodes exist, how many times they repeat, and which branch renders — with no imports required.

Key rules: 1. `@if` removes and recreates DOM nodes — it is not CSS `hide/show`; child `ngOnInit` fires each time 2. `@for` requires `track` — use a stable unique property (`item.id`), not `$index`, for dynamic lists 3. `@empty` pairs with `@for` — renders when the collection is empty, no separate `@if` needed 4. `@switch` uses strict equality (`===`) — type mismatches cause silent case-miss bugs 5. Built-in loop variables (`$index`, `$first`, `$last`, `$even`, `$odd`, `$count`) are available with no declaration 6. No `CommonModule` or `NgIf`/`NgFor` imports needed — the new syntax is compiler-native

The single most important Angular rule: Always track `item.id` in `@for` — track `$index` silently destroys and recreates every component when the list reorders, losing state and killing performance.

`@if` vs `[hidden]` — When to Use Which:

Criteria	<code>@if (cond)</code>	<code>[hidden]="!cond"</code>
DOM node when hidden	Removed	Present (<code>display: none</code>)
Component instance	Destroyed	Alive
<code>ngOnInit</code> on show	Fires each time	Fires once ever
Memory/CD cost when hidden	Zero	Full cost
State preserved when hidden	No	Yes

Default choice?

| Yes

| Only for state preservation

TOPIC 30 — Built-in Directives

(ngClass, ngStyle, ngTemplateOutlet)

Angular Core — Components & Architecture

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner-Intermediate — ngClass and ngStyle are straightforward to use but have multiple syntax forms that confuse developers, and knowing when not to use them (in favour of simpler bindings) requires experience. ngTemplateOutlet is a step up in abstraction and requires understanding Angular's template reference system.

Angular Relevance: High — ngClass and ngStyle appear in virtually every real Angular app. ngTemplateOutlet is essential for reusable component patterns, dynamic UI, and any scenario where you need to pass a template as a parameter.

Section 1 — Explanation

THE WARDROBE ANALOGY

ngClass is like a smart wardrobe assistant. You hand it a list of outfit rules — "wear the red jacket if it's cold, wear the formal shoes if there's a meeting, wear the hat if it's raining." It looks at all the conditions at once and puts on every item whose condition is true — multiple items simultaneously, each independently gated.

ngStyle is the same assistant but for fine-grained tailoring — "set the jacket length to exactly 72cm, set the collar width based on today's temperature." You're not toggling items on/off, you're setting precise values dynamically.

ngTemplateOutlet is a director calling an actor onto a stage from backstage. The actor (an ng-template) is defined somewhere in your template but isn't rendered yet — it's waiting in the wings. ngTemplateOutlet says "bring that actor on now" and optionally hands them a script (a context object with data). The same actor can be called onto multiple stages simultaneously with different scripts each time.

TECHNICAL EXPLANATION

These are Angular's three most commonly used attribute directives. Unlike structural directives (@if, @for) which add/remove DOM nodes, attribute directives modify the behaviour or appearance of an existing element.

The key distinction: - Structural directives (@if, @for) — change the DOM structure (add/remove nodes) - Attribute directives (ngClass, ngStyle) — change the DOM appearance (modify existing nodes) - ngTemplateOutlet is a structural directive that stamps a defined template into the DOM

ngClass — Dynamic CSS Classes

Applies one or more CSS classes based on conditions. Accepts three input forms:

Input Form	Syntax	Use When
String	<code>[ngClass]='class-a class-b'</code>	Adding static classes (prefer <code>class=</code> instead)
Array	<code>[ngClass]='class-a', 'class-b']</code>	List of class names to always apply
Object	<code>[ngClass]='{ class-a: condA }'</code>	Most common – conditional classes

The object form is by far the most useful: keys are class names, values are boolean expressions. Every key whose value is truthy gets applied as a class; falsy keys are removed.

ngStyle — Dynamic Inline Styles

Applies one or more inline styles dynamically. Accepts an object where keys are CSS property names and values are the style values.

```
[ngStyle]='{ color: textColor, font-size.px: fontSize }'
```

The `.px`, `.em`, `.%` unit suffixes are a convenience: `font-size.px: 14` is equivalent to `font-size: '14px'`.

ngTemplateOutlet — Dynamic Template Stamping

Renders a named ng-template reference at a specific location in the DOM, optionally passing data into it via a context object.

```
<ng-container *ngTemplateOutlet="myTemplate; context: { $implicit: item, label: 'Hello' }">
</ng-container>
```

- `$implicit` is the default context variable — accessed as the template's `let-item` variable - Named context properties are accessed via `let-label="label"` in the template declaration

Section 2 — Connection to Prior Concepts

Direct dependencies: - Topic 28 — Angular Template Syntax: `[ngClass]` and `[ngStyle]` are property bindings — they use `[squareBracket]` syntax - Topic 29 — Built-in Control Flow: `[class.x]="bool"` (single-class binding) and `@if` can sometimes substitute for `ngClass` - Topic 16 — TypeScript Generics: `ngTemplateOutlet` context objects benefit from typing with generics - Topic 13 — TypeScript Interfaces: Defining context types for `ngTemplateOutlet` uses interfaces - Topic 26 — `@Input()` & `@Output()`: `ngTemplateOutlet` is used heavily inside components that accept `TemplateRef` as an `@Input()`

Bridging note: In Topic 28 you learned `[class.active]="isActive"` for toggling a single class and `[style.color]="color"` for a single inline style. `ngClass` and `ngStyle` are the multi-property versions of exactly those bindings — same direction (class -> template), same square-bracket syntax, just capable of handling many classes or styles at once through an object map.

Most directly extends: Topic 28 — Angular Template Syntax

Section 3 — Code Example

PLAIN JS — THE CORE CONCEPT IN ISOLATION

```

const el = document.getElementById('card');

// ngClass equivalent – manually syncing classes from a conditions object
function applyClasses(element, classMap) {
  Object.entries(classMap).forEach(([className, condition]) => {
    if (condition) {
      element.classList.add(className);
    } else {
      element.classList.remove(className);
    }
  });
}

applyClasses(el, {
  'active': user.isActive,
  'admin': user.role === 'admin',
  'highlighted': user.id === selectedId
});

// ngStyle equivalent
function applyStyles(element, styleMap) {
  Object.entries(styleMap).forEach(([prop, value]) => {
    element.style[prop] = value;
  });
}

applyStyles(el, {
  color: user.isActive ? 'green' : 'grey',
  fontSize: `${user.priority * 4}px`
});

```

REAL ANGULAR EXAMPLE — ngClass AND ngStyle

```

import { NgClass, NgStyle } from '@angular/common';

@Component({
  selector: 'app-task-board',
  standalone: true,
  imports: [NgClass, NgStyle],
  template: `
    @for (task of tasks(); track task.id) {

      <!-- ngClass OBJECT FORM (most common) -->
      <div
        [ngClass]="{
          'task-card': true,
          'task-selected': task.isSelected,
          'task-blocked': task.status === 'blocked',
          'task-done': task.status === 'done',
          'priority-high': task.priority === 'high' || task.priority === 'critical',
          'priority-critical': task.priority === 'critical'
        }"
        (click)="selectTask(task.id)"
      >
        <h3>{{ task.title }}</h3>

      <!-- ngStyle OBJECT FORM -->

```

```

    <div
      class="progress-bar"
      [ngStyle]="{
        'width.%': task.progress,
        'background-color': getProgressColour(task.progress),
        'transition': 'width 0.3s ease'
      }"
    ></div>

    <span>{{ task.progress }}% complete</span>

    <!-- ngClass ARRAY FORM -->
    <span [ngClass]="getPriorityClasses(task.priority)">
      {{ task.priority }}
    </span>
  </div>
}
,
})
export class TaskBoardComponent {
  tasks = signal<Task[]>([...]);

  getPriorityClasses(priority: Task['priority']): string[] {
    const base = ['badge', `badge-${priority}`];
    if (priority === 'critical') base.push('badge-pulse');
    return base;
  }

  getProgressColour(progress: number): string {
    if (progress < 30) return '#ef4444';
    if (progress < 70) return '#f59e0b';
    return '#22c55e';
  }
}

```

REAL ANGULAR EXAMPLE — ngTemplateOutlet

```

import { NgTemplateOutlet } from '@angular/common';

@Component({
  selector: 'app-product-showcase',
  standalone: true,
  imports: [NgTemplateOutlet],
  template: `
    <!-- STEP 1: DEFINE the template – it is NOT rendered here -->
    <ng-template #productCard let-product let-featured="featured">
      <div [class.featured]="featured" class="product-card">
        <h3>{{ product.name }}</h3>
        <p>{{ product.price | currency }}</p>
        @if (product.badge) {
          <span class="badge">{{ product.badge }}</span>
        }
        @if (featured) {
          <div class="featured-ribbon">Featured</div>
        }
      </div>
    </ng-template>

    <!-- STEP 2: STAMP the template at this location -->

```

```

<section>
  <h2>All Products</h2>
  @for (product of products(); track product.id) {
    <ng-container
      *ngTemplateOutlet="productCard; context: { $implicit: product, featured: false }"
    ></ng-container>
  }
</section>

<!-- SAME TEMPLATE, DIFFERENT CONTEXT -->
<section>
  <h2>Featured Products</h2>
  @for (product of featuredProducts(); track product.id) {
    <ng-container
      *ngTemplateOutlet="productCard; context: { $implicit: product, featured: true }"
    ></ng-container>
  }
</section>

<!-- CONDITIONAL TEMPLATE SELECTION -->
<ng-template #compactView let-product>
  <span class="compact">{{ product.name }} -- {{ product.price | currency }}</span>
</ng-template>

<ng-template #detailedView let-product>
  <div class="detailed">
    <h4>{{ product.name }}</h4>
    <p>Price: {{ product.price | currency }}</p>
  </div>
</ng-template>

<div class="view-toggle">
  <button (click)="viewModel.set('compact')">Compact</button>
  <button (click)="viewModel.set('detailed')">Detailed</button>
</div>

@for (product of products(); track product.id) {
  <ng-container
    *ngTemplateOutlet="
      viewModel() === 'compact' ? compactView : detailedView;
      context: { $implicit: product }
    "
  ></ng-container>
}

})
export class ProductShowcaseComponent {
  viewModel = signal<'compact' | 'detailed'>('detailed');
  products = signal<Product[]>([...]);
  featuredProducts = signal<Product[]>([...]);
}

```

ngTemplateOutlet AS A COMPONENT @Input() — THE POWER PATTERN

```

// A reusable list component that accepts a template for each row
@Component({
  selector: 'app-data-list',
  standalone: true,

```

```

imports: [NgTemplateOutlet],
template: `
  <ul class="data-list">
    @for (item of items; track $index) {
      <li>
        @if (itemTemplate) {
          <ng-container
            *ngTemplateOutlet="itemTemplate; context: { $implicit: item, index: $index }"
          ></ng-container>
        } @else {
          <span>{{ item }}</span>
        }
      </li>
    }
  </ul>
`
})
export class DataListComponent<T> {
  @Input() items: T[] = [];
  @Input() itemTemplate?: TemplateRef<{ $implicit: T; index: number }>;
}

// Parent usage:
@Component({
  template: `
    <ng-template #userRow let-user let-i="index">
      <div class="user-row">
        {{ i + 1 }}. {{ user.name }} -- {{ user.email }}
      </div>
    </ng-template>

    <app-data-list
      [items]="users"
      [itemTemplate]="userRow"
    ></app-data-list>
  `
})
export class ParentComponent {
  users = [
    { name: 'Alice', email: 'alice@example.com' },
    { name: 'Bob', email: 'bob@example.com' },
  ];
}

```

Section 4 — Before & After Refactor

BEFORE — VERBOSE CONDITIONAL CLASS LOGIC

```

@Component({
  template: `
    <!-- String concatenation – brittle, hard to read, error-prone -->
    <div class="{{ 'card' + (isActive ? ' active' : '') + (isSelected ? ' selected' : '') + (has
      {{ title }}
    </div>

    <!-- Multiple [class.x] bindings pile up fast -->

```

```

<div class="card"
  [class.active]="isActive"
  [class.selected]="isSelected"
  [class.error]="hasError"
  [class.featured]="isFeatured"
  [class.disabled]="isDisabled"
  [class.loading]="isLoading">
  {{ title }}
</div>

<!-- Inline style concatenation – unreadable and type-unsafe -->
<div [style]="'color:' + textColor + ';font-size:' + fontSize + 'px'">
</div>
,
})

```

AFTER — CLEAN ngClass AND ngStyle

```

import { NgClass, NgStyle } from '@angular/common';

@Component({
  standalone: true,
  imports: [NgClass, NgStyle],
  template: `
    <!-- ngClass object: declarative, readable, independently toggleable -->
    <div
      [ngClass]="{
        'card': true,
        'active': isActive,
        'selected': isSelected,
        'error': hasError,
        'featured': isFeatured,
        'disabled': isDisabled,
        'loading': isLoading
      }"
    >
      {{ title }}
    </div>

    <!-- ngStyle object: each property independently declared and typed -->
    <div
      [ngStyle]="{
        'color': textColor,
        'font-size.px': fontSize,
        'opacity': isVisible ? 1 : 0
      }"
    ></div>
  `,
})

```

Why this matters in Angular: The object form co-locates all class logic in one binding. Angular's change detection efficiently diffs the object and only touches the specific classes that changed.

Section 5 — Common Mistakes & Misconceptions

MISTAKE 1 — Using ngStyle for Static or Rarely-Changing Styles

What the mistake looks like:

```
<div [ngStyle]="{ 'background-color': '#3b82f6', 'border-radius': '8px', 'padding': '16px' }">
```

Why developers make it: They learn `ngStyle` exists and default to it for any style — including static ones.

How to avoid it: Reserve `ngStyle` for styles whose values change at runtime based on component state. Static styles belong in the component's CSS file.

Angular-specific consequence: `ngStyle` runs on every change detection cycle, evaluating the object and syncing any changed properties. For purely static styles, this is unnecessary work.

MISTAKE 2 — Forgetting to Import `NgClass` / `NgStyle` in Standalone Components

What the mistake looks like:

```
@Component({
  standalone: true,
  imports: [], // NgClass/NgStyle not imported
  template: `<div [ngClass]="{ active: isActive }">...</div>`
  // Template compiles but [ngClass] silently does nothing
})
```

Why developers make it: The new built-in control flow (`@if`, `@for`) needs no imports, so developers assume `ngClass` and `ngStyle` also don't.

How to avoid it: Mental checklist: `@if/@for/@switch` = no import.

`ngClass/ngStyle/ngTemplateOutlet` = must import from `@angular/common`.

Angular-specific consequence: No console error — the binding silently fails. Classes are never applied, and the bug looks identical to a CSS issue.

MISTAKE 3 — Misunderstanding `$implicit` vs Named Context in `ngTemplateOutlet`

What the mistake looks like:

```
<!-- Passing context with a named property -->
<ng-container *ngTemplateOutlet="myTpl; context: { user: currentUser }">
</ng-container>

<!-- Template trying to access it as $implicit - gets undefined -->
<ng-template #myTpl let-user>
  {{ user?.name }} <!-- undefined - user was not $implicit -->
</ng-template>
```

Why developers make it: `let-user` looks like it should capture anything named `user`.

How to avoid it: Two rules: - `let-x` (no `=`) captures `$implicit` - `let-x="key"` captures the named key from context

Angular-specific consequence: The template renders with undefined data, producing blank UI with no error.

Section 6 — Do's and Don'ts

DO: - Use [ngClass] object form for multiple conditional classes - Use [class.x]="bool" (single-class binding) when toggling just one class - Import NgClass, NgStyle, NgTemplateOutlet explicitly in standalone components - Use ngStyle only for truly dynamic values (computed at runtime) - Use ng-container as the host for *ngTemplateOutlet — it adds no DOM node - Type your ngTemplateOutlet context with an interface for IDE support

DON'T: - Build class strings with string concatenation and ternary operators - Reach for ngClass for a single class — [class.active]="isActive" is simpler - Assume they work without imports like the built-in control flow blocks - Use ngStyle for static styles — put those in your component's CSS file - Use a real element like <div> as the host — it adds an unnecessary wrapper to the DOM - Leave context objects untyped — you lose autocomplete inside the template

Section 7 — Debugging Tips

BUG 1 — ngClass Object Applied But Classes Don't Appear in the DOM

Symptom: The [ngClass] binding is set, conditions look correct, but inspecting the element in DevTools shows none of the expected classes.

Cause (most common): NgClass is not in the component's imports array. The binding is silently ignored.

Fix:

```
import { NgClass } from '@angular/common';
@Component({
  standalone: true,
  imports: [NgClass], // Add this
  template: `<div [ngClass]="{ active: isActive }"></div>`
})
```

Angular-specific note: Also check that you're using [ngClass] (property binding) not ngClass="..." (attribute — always passes a string).

BUG 2 — ngTemplateOutlet Renders Nothing or Shows Blank Content

Symptom: The ng-container with *ngTemplateOutlet is in the DOM, but nothing renders inside it.

Cause: Either (a) the template reference variable name is wrong (typo in #templateName), (b) NgTemplateOutlet is not imported, or (c) the template reference isn't resolved.

Fix:

```
// Check 1: Import NgTemplateOutlet
import { NgTemplateOutlet } from '@angular/common';
@Component({ imports: [NgTemplateOutlet] })

// Check 2: Verify template reference name matches exactly
// <ng-template #myCard> ... </ng-template>
// *ngTemplateOutlet="myCard" must match exactly, case-sensitive
```

Angular-specific note: Add a temporary `{{ myCard }}` in the template — if it logs `[object Object]` it means the `TemplateRef` is resolved correctly.

BUG 3 — ngStyle Pixel Values Not Applied Correctly

Symptom: You use `[ngStyle]="{ 'font-size': fontSize }"` where `fontSize` is a number (e.g. 16), and the style either doesn't apply or applies as `font-size: 16` (no unit).

Cause: CSS requires units for most length properties. Passing a bare number (16) sets the style to "16" — which is invalid CSS and ignored by the browser.

Fix:

```
<!-- Option 1: Unit suffix syntax – Angular appends 'px' automatically -->
[ngStyle]="{ 'font-size.px': fontSize }"

<!-- Option 2: Concatenate the unit in the value -->
[ngStyle]="{ 'font-size': fontSize + 'px' }"
```

Angular-specific note: Unit suffixes work in `ngStyle` for `px`, `em`, `rem`, `%`, `vh`, `vw`.

Section 8 — Real-World Applications

SCENARIO 1 — Status-Driven Table Row Styling

```
@Component({
  selector: 'app-orders-table',
  standalone: true,
  imports: [NgClass, NgStyle, CurrencyPipe],
  template: `
    <table class="orders-table">
      <tbody>
        @for (order of orders; track order.id) {
          <tr
            [ngClass]="{
              'row-priority': order.isPriority,
              'row-cancelled': order.status === 'cancelled',
              'row-overdue': order.daysOpen > 7 && order.status === 'pending',
              'row-complete': order.status === 'delivered'
            }"
          >
            <td>{{ order.id }}</td>
            <td>{{ order.customer }}</td>
            <td
              [ngStyle]="{
                'font-weight': order.total > 1000 ? 'bold' : 'normal',
                'color': order.total > 5000 ? '#7c3aed' : 'inherit'
              }"
            >
              {{ order.total | currency }}
            </td>
            <td>
              <span [ngClass]="['status-badge', 'status-' + order.status]">
                {{ order.status }}
              </span>
            </td>
          </tr>
        }
      </tbody>
    </table>
  `
})
```

```

        </td>
      </tr>
    }
  </tbody>
</table>
,
}))

```

SCENARIO 2 — Reusable Card Component With Slotted Templates

```

@Component({
  selector: 'app-generic-card',
  standalone: true,
  imports: [NgTemplateOutlet, NgClass],
  template: `
    <div [ngClass]="['card', 'card-' + variant]">
      <div class="card-header">
        @if (headerTemplate) {
          <ng-container *ngTemplateOutlet="headerTemplate"></ng-container>
        } @else {
          <h3>{{ defaultTitle }}</h3>
        }
      </div>
      <div class="card-body">
        @if (bodyTemplate) {
          <ng-container
            *ngTemplateOutlet="bodyTemplate; context: { $implicit: data }"
          ></ng-container>
        }
      </div>
      @if (footerTemplate) {
        <div class="card-footer">
          <ng-container *ngTemplateOutlet="footerTemplate"></ng-container>
        </div>
      }
    </div>
  `,
})
export class GenericCardComponent<T> {
  @Input() data?: T;
  @Input() variant: 'default' | 'compact' | 'featured' = 'default';
  @Input() defaultTitle = 'Card';
  @Input() headerTemplate?: TemplateRef<void>;
  @Input() bodyTemplate?: TemplateRef<{ $implicit: T }>;
  @Input() footerTemplate?: TemplateRef<void>;
}

```

Section 9 — Practice Exercises

EXERCISE 1 — Beginner Build a standalone `AlertBanner` component that accepts a type input of `'info' | 'success' | 'warning' | 'error'` and a message string input. Use `[ngClass]` to apply a base class `alert` plus a dynamic class `alert-info`, `alert-success`, `alert-warning`, or `alert-error` depending on the type. Use `[ngStyle]` to set the font-weight to bold when the type is `error` or `warning`. Display the message using interpolation.

EXERCISE 2 — Intermediate Build a ThemeCard component where the card's background colour, text colour, and border radius are all driven by component properties (bgColour, textColour, borderRadius). Use [ngStyle] for these dynamic styles. Add three preset theme buttons (Light, Dark, Vibrant) that set all three style properties at once. The active theme button should have an active class applied using [ngClass]. Ensure all static styles (padding, font family, box shadow) are in the component's CSS file, not in ngStyle.

EXERCISE 3 — Advanced Build a reusable SelectableList<T> component that accepts an items @Input(), a labelTemplate TemplateRef @Input(), and a detailTemplate TemplateRef @Input(). The list renders each item using labelTemplate via ngTemplateOutlet. When an item is clicked, it becomes "selected" and the detailTemplate expands below it (also via ngTemplateOutlet), receiving the selected item as \$implicit context. Use [ngClass] to style the selected row differently. In a parent component, use this list with two different data types (e.g. users and products) with entirely different label and detail templates to prove the component is truly generic.

Section 10 — Key Takeaways

Core Concepts: - ngClass — applies/removes multiple CSS classes based on a string, array, or object map; object form is most common and most maintainable - ngStyle — sets multiple inline styles dynamically; use .px/.em unit suffix syntax to avoid string concatenation for numeric values - ngTemplateOutlet — stamps a named ng-template into the DOM at a specific location, optionally with a context object; enables template reuse and dynamic template selection - ng-template — defines a template fragment that is never rendered until explicitly stamped - ng-container — a logical wrapper with no DOM footprint; ideal host for *ngTemplateOutlet - \$implicit — the default context key in ngTemplateOutlet; accessed via let-x (no equals sign) in the template declaration - All three require importing from @angular/common in standalone components

Quick Reference Table:

Directive	Import From	Input Type	Common Use
NgClass	@angular/common	string string[] object	Multiple conditional classes
NgStyle	@angular/common	{ [prop: string]: any }	Dynamic computed styles
NgTemplateOutlet	@angular/common	TemplateRef null	Template reuse, dynamic layout

When to use each class-binding approach:

Scenario	Use
One class, one condition	[class.active]="isActive"
Two or three classes	[class.a]="x" [class.b]="y"
Four or more conditional classes	[ngClass]="{ ... }"
Computed list of class names	[ngClass]="getClasses()"

Angular-Specific Rules: - NgClass, NgStyle, NgTemplateOutlet must be explicitly imported in standalone components - Never use ngStyle for static values — use the component CSS file - ng-container leaves no DOM trace — always prefer it over <div> as the *ngTemplateOutlet host -

let-x captures \$implicit; let-x="key" captures a named context property

One-Line Memory Aid: ngClass toggles which classes exist, ngStyle controls what values styles have, and ngTemplateOutlet controls which template renders — three directives, three different dimensions of DOM control.

Section 11 — Thought-Provoking Question + Answer

Question: ngTemplateOutlet lets a parent pass a template into a child component. Angular also has ng-content (Topic 31) which lets a parent project HTML into a child. They seem to solve the same problem — putting parent-defined content inside a child component. What's the actual difference, and when does each pattern become the wrong choice?

Answer:

They look similar from the outside — both put parent-defined content inside a child component — but they operate at completely different levels of the rendering pipeline and have fundamentally different capabilities.

Conceptual explanation: ng-content is slot projection — it takes a chunk of already-rendered DOM from the parent and moves it into the child's template at the ng-content slot. The content is rendered in the parent's context — it sees the parent's variables.

ngTemplateOutlet with a TemplateRef input is template delegation — the parent defines a blueprint (an unrendered ng-template), passes the blueprint to the child, and the child stamps it wherever it wants, as many times as it wants, with whatever context data it chooses.

The critical difference: ng-content cannot receive data from the child component. The parent's projected content can only see parent-scope variables. ngTemplateOutlet with context can — this is how you build APIs where the child renders a list and the parent defines how each row looks, while each row receives the child's data.

When each becomes wrong: - ng-content is wrong when your parent-defined content needs access to data that only the child has - ngTemplateOutlet is wrong when the content is simple and static — it adds cognitive overhead for no benefit

Practical rule: If the parent's content needs data from the child to render correctly, use TemplateRef + ngTemplateOutlet. If the parent's content is self-contained and just needs a slot to live in, use ng-content.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts: - Computed property names — { [className]: condition } uses computed keys; understanding this JavaScript pattern makes the ngClass object form feel natural

TypeScript Concepts: - Generic types with constraints — typing TemplateRef<{ \$implicit: T }> for typed template contexts uses generics - Mapped types — useful for typing ngStyle objects and ngClass condition maps in strict codebases

Angular-Specific Concepts: - Topic 31 — Content Projection (ng-content) — the companion pattern to ngTemplateOutlet - Topic 41 — Custom Directives — ngClass and ngStyle are themselves attribute directives; building custom ones follows the same model - Topic 42 — Custom Pipes — pipes are often used inside ngClass expressions and ngTemplateOutlet contexts - Topic 32 — ViewChild, ViewChildren, ContentChild — accessing TemplateRef instances programmatically from @ViewChild enables dynamic ngTemplateOutlet patterns

Section 13 — Study Plan Checkpoint

This topic directly depends on: - Topic 13 — TypeScript Interfaces (typing context objects for ngTemplateOutlet) - Topic 16 — TypeScript Generics (typed TemplateRef<T> inputs) - Topic 26 — @Input() & @Output() (TemplateRef passed as @Input()) - Topic 28 — Angular Template Syntax (all three directives are property bindings) - Topic 29 — Built-in Control Flow (@if/@for used alongside these directives)

This topic unlocks: - Topic 31 — Content Projection — directly contrasts with ngTemplateOutlet - Topic 32 — ViewChild / ContentChild — accessing TemplateRef and ElementRef programmatically - Topic 41 — Custom Directives — ngClass and ngStyle are attribute directives; building custom ones follows the same pattern - Topic 45 — Performance Optimisation — understanding when ngClass object evaluation is expensive vs a computed signal

Production readiness assessment: You are ready to use ngClass and ngStyle confidently in production today. Use ngTemplateOutlet for straightforward template reuse, but practise the TemplateRef as @Input() pattern in a real component before shipping it — the \$implicit vs named context distinction trips up even experienced developers.

Section 14 — Common Interview Questions

QUESTION 1 — Junior-Mid "What are the different ways you can pass classes to an element in Angular? When would you use [class.x] versus [ngClass]?"

Strong answer covers: - Four approaches: class="static", [class.x]="bool", [ngClass]="object/array/string", [class]="expression" - [class.x] is ideal for a single conditional class — simple, readable, zero overhead - [ngClass] with an object is ideal for three or more classes with independent conditions - [ngClass] with an array is ideal when class names are computed programmatically - Both approaches can coexist on the same element: class="base-class" [ngClass]="{ active: isActive }" - NgClass must be imported in standalone components

Weak answers miss: That [class.x] and [ngClass] can be combined on the same element.

QUESTION 2 — Mid-Senior "How does ngTemplateOutlet work, and what is the purpose of the context object? What is \$implicit?"

Strong answer covers: - `ngTemplateOutlet` renders a `ng-template` reference at a specific DOM location - Templates defined with `<ng-template #ref>` are not rendered until explicitly stamped - The context object passes data into the template at render time - `$implicit` is the default context key — accessed in the template via `let-varName` (no equals sign) - Named properties require `let-varName="keyName"` syntax - The same template can be stamped multiple times with different contexts - `TemplateRef` can be passed as `@Input()` enabling truly generic, reusable components

Weak answers miss: The `$implicit` vs named context distinction, and the `TemplateRef` as `@Input()` pattern.

QUESTION 3 — Mid "What is `ng-container` and why do developers use it as the host for `*ngTemplateOutlet`?"

Strong answer covers: - `ng-container` is a grouping element that Angular removes from the DOM after compilation — it leaves no DOM trace - Using a `<div>` or `<span>` as the host for `*ngTemplateOutlet` adds an extra wrapper element to the DOM - This extra wrapper can break CSS layouts (flexbox/grid, table structure, list structure) - `ng-container` is also used with structural directives when you need to apply the directive without adding a wrapper element - Common use: `@for` over `<tr>` rows in a table — wrapping in `<div>` would break the table; `ng-container` does not

Weak answers miss: The CSS layout breakage consequence.

Section 15 — Anti-Patterns to Avoid in Production

ANTI-PATTERN 1 — Computing `ngClass` Objects Inside the Template Expression

```
<!-- Object literal constructed on every change detection cycle -->
<div
  [ngClass]="{
    'active': userService.getCurrentUser()?.isActive,
    'premium': subscriptionService.getTier() === 'premium',
    'verified': verificationService.check(userId)
  }"
>
```

Why it seems fine: It reads declaratively and produces the correct visual output.

What breaks in production: Three service method calls execute on every change detection cycle. With default change detection and a complex page, this can be hundreds of calls per second.

Correct pattern:

```
cardClasses = computed(() => ({
  'active': this.userService.currentUser()?.isActive ?? false,
  'premium': this.subscriptionService.tier() === 'premium',
  'verified': this.isVerified()
}));
// Template: <div [ngClass]="cardClasses()"></div>
```

ANTI-PATTERN 2 — Using `ngStyle` for Theme/Brand Styling


```
<div [ngStyle]="{ 'background': theme.primary, 'color': theme.text }">
  <h1 [ngStyle]="{ 'font-family': theme.fontFamily }">Title</h1>
  <!-- Repeated across 50+ elements in the component -->
</div>
```

Why it seems fine: Dynamic theming is a real requirement and `ngStyle` achieves the goal.

What breaks in production: Fifty `[ngStyle]` bindings checked on every change detection cycle. No CSS caching. Dark mode can't be toggled with a CSS class.

Correct pattern:

```
ngOnInit() {
  // Set CSS custom properties on the root element once
  this.el.nativeElement.style.setProperty('--primary', this.theme.primary);
  this.el.nativeElement.style.setProperty('--text', this.theme.text);
}
// CSS: .themed-container { background: var(--primary); color: var(--text); }
```

ANTI-PATTERN 3 — Defining `ng-template` Inside `@for` Loops

```
@for (item of items; track item.id) {
  <ng-template #rowTemplate let-data>
    <!-- This template is re-defined on every loop iteration -->
    <span>{{ data.label }}</span>
  </ng-template>

  <ng-container *ngTemplateOutlet="rowTemplate; context: { $implicit: item }">
  </ng-container>
}
```

Why it seems fine: It's co-located with its usage and produces the right output for simple cases.

What breaks in production: A new `TemplateRef` instance is created for every loop iteration. The `#rowTemplate` reference is overwritten on each iteration. Behaviour is undefined and depends on evaluation order.

Correct pattern:

```
<!-- Define the template ONCE, outside the loop -->
<ng-template #rowTemplate let-data>
  <span>{{ data.label }}</span>
</ng-template>

@for (item of items; track item.id) {
  <ng-container *ngTemplateOutlet="rowTemplate; context: { $implicit: item }">
  </ng-container>
}
```

Section 16 — Mental Model Summary

Core concept: `ngClass` and `ngStyle` are multi-property versions of Angular's single-property class and style bindings — both are attribute directives that modify an existing element.

`ngTemplateOutlet` is a structural directive that stamps a defined `ng-template` blueprint into the

DOM at a specific location, optionally with injected context data.

Key rules: 1. NgClass, NgStyle, NgTemplateOutlet — must be imported from @angular/common in standalone components 2. Use [class.x]="bool" for one class; use [ngClass]="object" for three or more 3. Use ngStyle only for runtime-computed values — static styles belong in CSS 4. In ngTemplateOutlet context: let-x captures \$implicit; let-x="key" captures a named key 5. Always host *ngTemplateOutlet on <ng-container> — it leaves no DOM footprint 6. Define <ng-template> outside loops — inside loops creates a new TemplateRef per iteration

The single most important Angular rule: NgClass, NgStyle, and NgTemplateOutlet are not built into the compiler — unlike @if/@for, they will silently do nothing if you forget to add them to imports: [].

Choosing the Right Class Binding:

Conditions	Use	Example
-----	-----	-----
1 class, 1 condition	[class.x]	[class.active]="isOn"
2-3 classes	Multiple [class.x]	[class.a]="x" [class.b]="y"
4+ conditional classes	[ngClass] object	[ngClass]="{ a: x, b: y }"
Computed class list	[ngClass] array	[ngClass]="getClasses()"

ng-content vs ngTemplateOutlet:

Criteria	ng-content	ngTemplateOutlet
-----	-----	-----
Parent provides	Rendered HTML	Unrendered blueprint
Child can pass data back	No	Yes (via context)
Stamped how many times	Once	As many times as needed
Complexity	Low	Medium
Best for	Simple slot projection	Data-aware row/cell templates

TOPIC 31 — Content Projection

(ng-content)

Angular Core — Components & Architecture

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The basic single-slot pattern is simple, but multi-slot projection with `select`, content projection combined with `@ContentChild`, and understanding which context projected content renders in trips up even experienced developers.

Angular Relevance: High — Content projection is the mechanism behind every reusable container component in Angular — cards, modals, tabs, accordions, layout wrappers. Without it, you cannot build a proper component library or shared UI layer.

Section 1 — Explanation

THE PICTURE FRAME ANALOGY

Imagine a picture frame manufacturer. The manufacturer builds beautiful frames — ornate borders, glass, backing, hanging hardware. But the manufacturer has no idea what picture will go inside. The frame has a slot where a picture goes, and whoever buys the frame provides their own picture.

`ng-content` is that slot. Your component is the frame — it defines structure, styling, and behaviour. The parent provides the picture — the content that goes inside. The frame doesn't care what the picture is. The picture doesn't care what frame it's in. They combine cleanly because the slot is the contract between them.

A multi-slot frame is like a shadow box with labelled compartments — "photo goes here, caption goes here, date stamp goes here." The `select` attribute on `ng-content` is the label on each compartment.

TECHNICAL EXPLANATION

Content projection is Angular's mechanism for allowing a parent component to pass HTML (including other components) into a child component's template. The child defines where that content appears using `<ng-content>`. The parent defines what that content is.

Without content projection, every container component (card, modal, panel) would need `@Input()` properties for every piece of content — `titleInput`, `bodyInput`, `footerInput` — all as strings. That kills rich HTML content, nested components, and event bindings.

Three patterns of increasing power:

1 — Single-slot projection (one `<ng-content>`) All content passed by the parent lands in the one slot.

2 — Multi-slot projection (<ng-content select="...">) Different pieces of parent content are routed to different named slots using CSS selectors.

3 — Conditional projection (<ng-content> inside @if) The slot can be conditionally included — but with an important caveat: the content is always instantiated even if the slot is not rendered.

The select attribute accepts any CSS selector:

Selector Type	Example	Matches
Element	select="header"	<header> elements
Attribute	select="[slot=footer]"	Elements with slot="footer"
CSS class	select=".card-title"	Elements with class card-title
Component	select="app-icon"	<app-icon> component elements

THE RENDERING CONTEXT RULE — the most important concept in content projection:

Projected content is compiled in the parent's context and rendered in the child's slot.

This means: - Variables, methods, signals in {{ }} inside projected content refer to the parent component's class - CSS styles from the child component's stylesheet apply to the projected content's slot position but not to the content itself - The parent's change detection governs the projected content's bindings

Section 2 — Connection to Prior Concepts

Direct dependencies: - Topic 28 — Angular Template Syntax: projected content uses all the same binding syntax — {{ }}, [prop], (event) — but those bindings resolve against the parent component - Topic 29 — Built-in Control Flow: @if and @for inside projected content work normally, governed by the parent - Topic 30 — Built-in Directives: ngClass, ngStyle, ngTemplateOutlet all work inside projected content - Topic 25 — Component Lifecycle Hooks: projected content's component lifecycle hooks fire at the child's mount time, not the parent's - Topic 26 — @Input() & @Output(): content projection is an alternative to @Input() for passing rich content into components - Topic 32 — ViewChild / ContentChild (upcoming): @ContentChild and @ContentChildren are how the child component queries its projected content

Bridging note: In Topic 30 you saw that ngTemplateOutlet with a TemplateRef @Input() lets a parent pass a template blueprint into a child. Content projection with ng-content is a simpler, more declarative version of the same idea — instead of explicitly passing a TemplateRef, you just write HTML between the component's opening and closing tags, and Angular routes it to the slot. Topic 30's TemplateRef pattern gives you more control (the child stamps the template with context data); ng-content is simpler but the child cannot inject data back into the projected content.

Most directly extends: Topic 30 — Built-in Directives (ngTemplateOutlet)

Section 3 — Code Example

PLAIN JS — THE CORE CONCEPT IN ISOLATION

```
// Child "component" – defines a container with a slot
function createCard(projectedContent) {
  const card = document.createElement('div');
  card.className = 'card';

  const body = document.createElement('div');
  body.className = 'card-body';
  // The "slot" – parent provides this content
  body.appendChild(projectedContent);

  card.appendChild(body);
  return card;
}

// Parent – creates the content to project
const myContent = document.createElement('p');
myContent.textContent = 'This content is defined by the parent.';

// Parent passes its content into the child's slot
document.body.appendChild(createCard(myContent));

// Angular's ng-content does exactly this, but:
// - Declaratively in HTML (no JS needed)
// - With multi-slot routing via select=
// - With proper change detection for the projected bindings
// - With correct component context for those bindings
```

REAL ANGULAR EXAMPLE — SINGLE-SLOT PROJECTION

```
@Component({
  selector: 'app-card',
  standalone: true,
  template: `
    <div class="card">
      <div class="card-shell">
        <!-- ng-content is the slot. Whatever the parent puts between
              <app-card> and </app-card> lands here at runtime. -->
        <ng-content></ng-content>
      </div>
    </div>
  `,
  styles: [`.card { border: 1px solid #e2e8f0; border-radius: 8px; padding: 16px; }`]
})
export class CardComponent {}

// PARENT USAGE
@Component({
  selector: 'app-page',
  standalone: true,
  imports: [CardComponent],
  template: `
    <!-- Everything between <app-card> and </app-card> is projected content -->
    <!-- {{ userName }} resolves to the PARENT's userName property -->
    <app-card>
      <h3>Welcome back, {{ userName }}!</h3>
      <p>You have {{ messageCount }} unread messages.</p>
    </app-card>
  `
})
```

```

    <button (click)="viewMessages()">View Messages</button>
  </app-card>

  <!-- Same card component – completely different content -->
  <app-card>
    
    <p>Angular In Depth -- 49.99</p>
    <button (click)="addToCart()">Add to Cart</button>
  </app-card>
  ,
})
export class PageComponent {
  userName = 'Alice';
  messageCount = 4;
  viewMessages(): void { console.log('Viewing messages'); }
  addToCart(): void { console.log('Added to cart'); }
}

```

MULTI-SLOT PROJECTION WITH select

```

@Component({
  selector: 'app-panel',
  standalone: true,
  template: `
    <div class="panel">
      <!-- SLOT 1: matches elements with the panel-header attribute -->
      <div class="panel-header">
        <ng-content select="[panel-header]"></ng-content>
      </div>

      <!-- SLOT 2: matches elements with class panel-body -->
      <div class="panel-body">
        <ng-content select=".panel-body"></ng-content>
      </div>

      <!-- SLOT 3: matches a specific component -->
      <div class="panel-footer">
        <ng-content select="app-panel-footer"></ng-content>
      </div>

      <!-- SLOT 4: default (no select) – catches everything NOT matched above -->
      <div class="panel-default">
        <ng-content></ng-content>
      </div>
    </div>
  `
})
export class PanelComponent {}

// PARENT USAGE
@Component({
  template: `
    <app-panel>
      <!-- This h2 has panel-header attribute -> routes to SLOT 1 -->
      <h2 panel-header>Dashboard Overview</h2>

      <!-- This div has class panel-body -> routes to SLOT 2 -->
      <div class="panel-body">

```

```

    <p>Here is the main content. Messages: {{ count }}</p>
  </div>

  <!-- No matching selector -> routes to SLOT 4 (default) -->
  <p>This content has no selector.</p>
</app-panel>
,
})
export class ParentComponent {
  count = 12;
}

```

CONDITIONAL SLOT — THE INSTANTIATION CAVEAT

```

@Component({
  selector: 'app-collapsible',
  standalone: true,
  template: `
    <div class="collapsible">
      <button (click)="toggle()">{{ isOpen ? 'Hide' : 'Show' }}</button>

      <!-- IMPORTANT: Even when isOpen is false and ng-content is not rendered,
           the projected content IS still instantiated.
           Any ngOnInit in a projected component will fire regardless.
           Only the DISPLAY is conditional – not the component lifecycle. -->
      @if (isOpen) {
        <div class="content">
          <ng-content></ng-content>
        </div>
      }
    </div>
  `,
})
export class CollapsibleComponent {
  isOpen = false;
  toggle(): void { this.isOpen = !this.isOpen; }
}

```

Section 4 — Before & After Refactor

BEFORE — STRING @Input() FOR RICH CONTENT

```

@Component({
  selector: 'app-modal',
  standalone: true,
  template: `
    <div class="modal">
      <!-- Renders a string – no HTML, no components, no bindings -->
      <h2>{{ title }}</h2>
      <p>{{ body }}</p>
      <div class="modal-footer">{{ footer }}</div>
    </div>
  `,
})
export class ModalComponent {
  @Input() title = '';
}

```

```

    @Input() body = '';
    @Input() footer = '';
}

```

```

// Parent – forced to pass everything as plain strings
// <app-modal title="Confirm Delete" body="Are you sure?" footer="Click OK">

```

Problems: 1. Body can't contain an `<a>` link, a `<strong>` tag, or another component 2. Footer can't have a real `<button>` with a (click) binding 3. Every new content need requires a new `@Input()` on the component

AFTER — CONTENT PROJECTION

```

@Component({
  selector: 'app-modal',
  standalone: true,
  template: `
    <div class="modal-overlay" (click)="onOverlayClick()">
      <div class="modal" (click)="$event.stopPropagation()">
        <div class="modal-header">
          <ng-content select="[modal-header]"></ng-content>
          <button class="modal-close" (click)="close()">x</button>
        </div>
        <div class="modal-body">
          <ng-content select="[modal-body]"></ng-content>
        </div>
        <div class="modal-footer">
          <ng-content select="[modal-footer]"></ng-content>
        </div>
      </div>
    </div>
  `
})
export class ModalComponent {
  close(): void { /* emit close event */ }
  onOverlayClick(): void { this.close(); }
}

// Parent – passes rich content with real HTML and Angular bindings
@Component({
  template: `
    <app-modal>
      <h2 modal-header>
        Delete <strong>{{ itemName }}</strong>?
      </h2>
      <div modal-body>
        <p>This will permanently remove <em>{{ itemName }}</em>.</p>
        <p>{{ warningCount }} related records will also be deleted.</p>
        <a href="/help/delete" target="_blank">Learn more</a>
      </div>
      <div modal-footer>
        <button (click)="cancel()">Cancel</button>
        <button (click)="confirmDelete()" class="btn-danger">Delete</button>
      </div>
    </app-modal>
  `
})

```



```
export class PageComponent {
  itemName = 'Project Alpha';
  warningCount = 14;
  cancel(): void { }
  confirmDelete(): void { }
}
```

Why this matters in Angular: Content projection makes components composable. The modal knows nothing about what it contains — it provides structure, styling, overlay behaviour, and close logic. The parent provides the specific content for this use case.

Section 5 — Common Mistakes & Misconceptions

MISTAKE 1 — Assuming Projected Content Resolves Against the Child Component

What the mistake looks like:

```
@Component({
  selector: 'app-wrapper',
  template: `<ng-content></ng-content>`
})
export class WrapperComponent {
  childMessage = 'Hello from the child'; // Developer expects this to be accessible
}

// Parent template
@Component({
  template: `
    <app-wrapper>
      <!-- Developer expects childMessage to come from WrapperComponent -->
      <p>{{ childMessage }}</p> <!-- undefined – resolves against PARENT -->
    </app-wrapper>
  `
})
export class ParentComponent {
  // childMessage doesn't exist here – so {{ childMessage }} renders as blank
}
```

Why developers make it: The content visually appears inside the child component.

How to avoid it: Memorise the rule: "Projected content compiles in the parent, renders in the child." If you need the child's data inside projected content, use `ngTemplateOutlet` with a context object.

Angular-specific consequence: The binding silently renders as empty string — no error, no warning. One of the hardest bugs to spot.

MISTAKE 2 — Expecting `@if` Around `ng-content` to Delay Component Instantiation

What the mistake looks like:

```
// Developer expects app-heavy-chart to NOT initialise when isOpen = false
@Component({
  template: `
    @if (isOpen) {
```

```

        <ng-content></ng-content>
    }
    ,
  })
  export class LazyPanelComponent {
    isOpen = false;
  }

  // Parent: <app-lazy-panel><app-heavy-chart></app-heavy-chart></app-lazy-panel>
  // app-heavy-chart's ngOnInit fires immediately regardless

```

Why developers make it: `@if` around other components in a template does delay their initialisation.

How to avoid it: Projected content is part of the parent's template — it is instantiated when the parent mounts. The child's `@if` only controls whether the already-instantiated content is displayed.

The fix when lazy init is needed:

```

  // Use TemplateRef @Input() instead – the parent passes a blueprint, not live content
  @Input() contentTemplate?: TemplateRef<void>;
  // Template: @if (isOpen) { <ng-container *ngTemplateOutlet="contentTemplate"> }

```

Angular-specific consequence: `ngOnInit` on projected components fires immediately on parent mount. Any HTTP calls in those components runs regardless of the child's conditional display logic.

MISTAKE 3 — Using `ng-content` Inside `@for` (Attempting to Project Multiple Times)

What the mistake looks like:

```

@Component({
  selector: 'app-repeater',
  template: `
    @for (item of items; track item.id) {
      <!-- Can't work – ng-content projects the content ONCE, not per iteration -->
      <div class="item-wrapper">
        <ng-content></ng-content>
      </div>
    }
  `
})

```

Why developers make it: Seems logical — the child has a list of items, so stamp the projected template once per item.

How to avoid it: `ng-content` can only project content once. It moves the parent's DOM nodes into the slot — it cannot copy them. If you need to stamp a template once per data item, use `TemplateRef` + `ngTemplateOutlet`.

Angular-specific consequence: The projected content appears only once. The rest of the iterations render empty wrapper divs. This fails silently with no error.

Section 6 — Do's and Don'ts

DO: - Use `ng-content` for container components that provide structure but not specific content (cards, modals, panels, tabs) - Use `select` attribute for multi-slot projection — CSS attribute selectors (`[slot-name]`) are the most readable - Remember that projected content's bindings resolve against the parent component - Use `TemplateRef` + `ngTemplateOutlet` when you need the child to pass data back into the projected content - Add a default `<ng-content>` (no `select`) as a catch-all for unmatched content - Document your `select` attributes in comments — they are the public API of your component

DON'T: - Use `@Input()` title: string when the content needs to be rich HTML with components or event bindings - Use CSS class selectors for projection slots in large apps — classes collide; attribute selectors are explicit - Expect projected content to access the child component's properties or methods - Try to pass child data into `ng-content` projected content — it's impossible with basic projection - Let unmatched projected content disappear silently — users of your component will be confused - Leave `select` attributes undocumented

Section 7 — Debugging Tips

BUG 1 — Projected Content Disappears Entirely (No Slot Matches)

Symptom: You project content into a component, but it doesn't appear anywhere in the rendered output.

Cause: The component uses multi-slot projection with `select` attributes, and none of the parent's projected content matches any selector. Content that matches no slot is silently discarded if there is no default `<ng-content>`.

Fix:

```
// Add a default fallback slot to catch unmatched content during debugging
<ng-content></ng-content>  <!-- No select = catches everything unmatched -->

// Verify your selectors match
// If your slot is: <ng-content select="[panel-header]">
// Your projected content must have: <h2 panel-header>...</h2>
// NOT: <h2 class="panel-header"> (that's a class, not an attribute)
```

Angular-specific note: Attribute-based selectors (`[panel-header]`) are the most reliable.

BUG 2 — Styles From Child Component Not Applying to Projected Content

Symptom: You have styles in the child component's CSS file but projected elements from the parent are not styled.

Cause: Angular's view encapsulation (default: Emulated) scopes styles to the component that defines them. Projected content is defined in the parent's template — it gets the parent's style scope, not the child's.

Fix:

```
/* Option 1: In the child's CSS, use :ng-deep to pierce encapsulation */
:ng-deep .panel-body p { color: red; }
```

```
/* Option 2: Style the SLOT WRAPPER in the child */
.panel-body { color: red; } /* applies to wrapper, projected content inherits */

/* Option 3: Move styles to global stylesheet */
```

Angular-specific note: The cleanest solution is to style the slot wrapper element in the child's CSS — inherited CSS properties (color, font) cascade into projected content naturally.

BUG 3 — ngOnInit Fires in Projected Component Before Expected

Symptom: A projected component's ngOnInit runs immediately on page load, even though the slot it projects into is hidden behind an @if (false).

Cause: Projected content is part of the parent's template and is instantiated with the parent. The child's @if controls display, not instantiation.

Fix:

```
// Switch from content projection to TemplateRef input
// Parent changes:
// BEFORE: <app-panel><app-chart></app-chart></app-panel>
// AFTER:

<ng-template #chartTemplate>
  <app-chart></app-chart> <!-- Only instantiated when the child stamps this -->
</ng-template>
<app-panel [contentTemplate]="chartTemplate"></app-panel>

// In child component:
@Input() contentTemplate?: TemplateRef<void>;
// Template: @if (isOpen) { <ng-container *ngTemplateOutlet="contentTemplate"> }
```

Angular-specific note: This distinction — instantiation vs display — is critical for performance in tab components, accordions, and wizard flows.

Section 8 — Real-World Applications

SCENARIO 1 — Reusable Modal Component

```
@Component({
  selector: 'app-modal',
  standalone: true,
  template: `
    <div class="modal-overlay" (click)="handleOverlayClick()">
      <div class="modal-shell"
        [class.modal-sm]="size === 'sm'"
        [class.modal-lg]="size === 'lg'"
        (click)="$event.stopPropagation()">

        <div class="modal-header">
          <ng-content select="[modal-title]"></ng-content>
          <button class="modal-close" (click)="closed.emit()">x</button>
        </div>
        <div class="modal-body">
          <ng-content select="[modal-body]"></ng-content>
        </div>
      </div>
    </div>
```

```

        </div>
        <div class="modal-footer">
            <ng-content select="[modal-footer]"></ng-content>
        </div>
    </div>
</div>
,
})
export class ModalComponent {
    @Input() size: 'sm' | 'md' | 'lg' = 'md';
    @Input() closeOnOverlay = true;
    @Output() closed = new EventEmitter<void>();
    handleOverlayClick(): void {
        if (this.closeOnOverlay) this.closed.emit();
    }
}

// USAGE 1: Confirmation dialog
@Component({
    template: `
        @if (showConfirm) {
            <app-modal size="sm" (closed)="showConfirm = false">
                <h3 modal-title>Confirm Action</h3>
                <p modal-body>Are you sure you want to delete <strong>{{ selectedItem?.name }}</strong>?</p>
                <div modal-footer>
                    <button (click)="showConfirm = false">Cancel</button>
                    <button class="danger" (click)="confirmDelete()">Delete</button>
                </div>
            </app-modal>
        }
    `
})
export class DeletePageComponent {
    showConfirm = false;
    selectedItem: { name: string } | null = null;
    confirmDelete(): void { /* API call */ }
}

```

SCENARIO 2 — Tab Component With Projected Tab Panels

```

@Component({
    selector: 'app-tab-panel',
    standalone: true,
    template: `<ng-content></ng-content>`
})
export class TabPanelComponent {
    @Input() label = '';
    @Input() tabId = '';
}

@Component({
    selector: 'app-tabs',
    standalone: true,
    template: `
        <div class="tab-bar">
            @for (tab of tabPanels; track tab.tabId) {
                <button
                    [class.active]="activeTab() === tab.tabId"

```

```

        (click)="activeTab.set(tab.tabId)"
      >
        {{ tab.label }}
      </button>
    }
  </div>
  <div class="tab-content">
    <ng-content></ng-content>
  </div>
  ,
})
export class TabsComponent implements AfterContentInit {
  @ContentChildren(TabPanelComponent) tabPanels!: QueryList<TabPanelComponent>;
  activeTab = signal('');

  ngAfterContentInit(): void {
    if (this.tabPanels.length > 0) {
      this.activeTab.set(this.tabPanels.first.tabId);
    }
  }
}

// PARENT USAGE
@Component({
  template: `
    <app-tabs>
      <app-tab-panel label="Overview" tabId="overview">
        <h2>Project Overview</h2>
        <p>{{ projectDescription }}</p>
      </app-tab-panel>
      <app-tab-panel label="Settings" tabId="settings">
        <app-project-settings [projectId]="projectId"></app-project-settings>
      </app-tab-panel>
      <app-tab-panel label="Members" tabId="members">
        <app-team-list [projectId]="projectId"></app-team-list>
      </app-tab-panel>
    </app-tabs>
  `
})
export class ProjectPageComponent {
  projectId = 7;
  projectDescription = 'A collaborative workspace.';
}

```

Section 9 — Practice Exercises

EXERCISE 1 — Beginner Build a standalone `CardComponent` that uses a single `<ng-content>` slot. Add a fixed card shell with a border, padding, and box shadow defined in the component's CSS. In a parent component, use `CardComponent` three times with different projected content — a user profile summary, a weather widget, and a news headline. Verify that all three use the same card shell styling but have completely different inner content.

EXERCISE 2 — Intermediate Build a `SectionComponent` with three named `ng-content` slots using attribute selectors: `[section-title]`, `[section-body]`, and `[section-actions]`. The title slot should

render in a styled header bar, the body in a scrollable area, and the actions in a right-aligned footer. Add a default `<ng-content>` slot as a catch-all. In a parent, create two instances: one for a data summary section and one for a settings form section.

EXERCISE 3 — Advanced Build an `AccordionComponent` that accepts any number of projected `AccordionItemComponent` children. Each `AccordionItemComponent` takes a title `@Input()`. The accordion container reads the projected items' titles (hint: you'll need `@ContentChildren` from Topic 32 — use this exercise as a preview) and renders a clickable header for each. Only one item's `<ng-content>` body is visible at a time. Clicking a header expands its content and collapses the previously active one. Use CSS transitions for the open/close animation rather than `@if` — to demonstrate the difference between hiding and destroying projected content.

Section 10 — Key Takeaways

Core Concepts: - Content projection — allows a parent to inject HTML (including components) into a child component's template via `<ng-content>` slots - `<ng-content>` — the slot placeholder; replaced at runtime with the parent's projected content - select attribute — CSS selector that routes specific parent content to specific slots - Default slot — `<ng-content>` with no select; catches all content not matched by named slots - Rendering context rule — projected content's bindings resolve against the parent component's class, not the child's - Instantiation rule — projected content components are instantiated when the parent mounts, regardless of `@if` around `<ng-content>` in the child - `ng-content` cannot be repeated — unlike `ngTemplateOutlet`, it moves DOM nodes once

Quick Reference:

Pattern	Syntax	Use Case
Single slot	<code><ng-content></code>	Simple container components
Named slot	<code><ng-content select="[attr]"></code>	Multi-region layouts (modal, panel)
Default catch-all	<code><ng-content></code> after named	Catch unmatched content
Conditional display	<code>@if (open) { <ng-content> }</code>	Toggle visibility (NOT instantiation)
Repeated with data	<code>TemplateRef + ngTemplateOutlet</code>	Rows, cells, repeated items

Angular-Specific Rules: - Projected content bindings always resolve against the parent component — never the child - `@if` around `<ng-content>` controls display, not instantiation — projected components always initialise with the parent - select attribute selectors (`[slot-name]`) are more reliable than class selectors for projection routing - Unmatched projected content is silently discarded if no default `<ng-content>` exists - Child component CSS does not naturally style projected content

One-Line Memory Aid: The parent writes the picture, the child provides the frame — and the picture always uses the parent's paints, not the frame's.

Section 11 — Thought-Provoking Question + Answer

Question: Content projection and `ngTemplateOutlet` with `TemplateRef` inputs both let a parent define content that renders inside a child component. But they have a fundamental difference in

when and how content is instantiated. At what point in a real application does this difference force you to switch from ng-content to TemplateRef, and what does that tell you about the design contract each pattern creates?

Answer:

The forcing function is almost always one of two things: the child needs to pass data into the content or the child needs to control when the content is instantiated.

Conceptual explanation: ng-content represents a declarative slot contract — "put your content here; I'll display it." The child is passive. It receives already-live, already-bound DOM nodes and places them. It cannot inject data into those nodes, and it cannot delay their creation.

TemplateRef + ngTemplateOutlet represents an imperative template contract — "give me a blueprint; I'll decide when to build it and what data to give the builder." The child is active. It controls instantiation timing and can inject context data.

The two forcing functions in real apps:

1 — Data flows from child to template: A virtual scroll component knows which items are currently visible — the parent doesn't. For each visible item, the virtual scroller needs to render the parent's row template with that item's data. ng-content cannot do this. TemplateRef can — via context: { \$implicit: item }.

2 — Lazy instantiation: A tab component with ten heavy tabs using ng-content instantiates all ten panels at page load. With TemplateRef, the tab component only stamps a template when its tab is first activated — true lazy initialisation.

What this tells you about design contracts: - ng-content is a display contract: "I'll show your content in my layout" - TemplateRef is a rendering contract: "I'll render your template on my terms, with my data"

Practical rule: Start with ng-content — it's simpler, more readable, and sufficient for the majority of container components. Switch to TemplateRef when you hit either forcing function: the child needs to provide data to the template, or the child needs to control when the template is first rendered.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts: - WeakRef and object lifecycle — understanding JavaScript object instantiation helps clarify why projected content's components initialise with the parent

TypeScript Concepts: - Generic constraints — used when typing TemplateRef<T> context objects

Angular-Specific Concepts: - Topic 32 — ViewChild, ViewChildren, ContentChild — @ContentChild and @ContentChildren are how the child component queries what's been projected into it - Topic 41 — Custom Directives — attribute directives applied to projected content are a common pattern - Topic 47 — Testing Angular Components — testing projected

content requires specific TestBed setup

Section 13 — Study Plan Checkpoint

This topic directly depends on: - Topic 25 — Component Lifecycle Hooks (projection affects when `ngOnInit/ngOnDestroy` fire) - Topic 26 — `@Input()` & `@Output()` (projection is the alternative to `@Input()` for rich content) - Topic 28 — Angular Template Syntax (projected content uses all binding syntax, resolved in parent) - Topic 29 — Built-in Control Flow (`@if` inside projected content, `@if` around `ng-content`) - Topic 30 — Built-in Directives (`ngTemplateOutlet` is the `TemplateRef` alternative to `ng-content`)

This topic unlocks: - Topic 32 — `ViewChild`, `ViewChildren`, `ContentChild` — directly depends on this topic; `@ContentChild` queries projected content - Topic 41 — Custom Directives — directives applied to projected content behave according to projection's context rules - Topic 47 — Testing Angular Components — testing components with `ng-content` requires understanding the projection model

Production readiness assessment: You are ready to use single-slot and multi-slot `ng-content` projection confidently in production. Before shipping complex interactive containers (tabs, accordions, wizard flows), practise the instantiation-vs-display distinction and know when to switch to `TemplateRef`.

Section 14 — Common Interview Questions

QUESTION 1 — Junior-Mid "What is content projection in Angular and how does `ng-content` work? What does the `select` attribute do?"

Strong answer covers: - Content projection allows a parent to pass HTML/components into a child's template via `<ng-content>` slots - The child defines where content appears; the parent defines what content is - `select` accepts a CSS selector to route specific parent content to specific slots - Content not matching any `select` goes to the default `<ng-content>` (no `select`) - Content not matched by any slot and with no default slot is silently discarded

Weak answers miss: What happens to unmatched content (silent discard).

QUESTION 2 — Mid-Senior "In Angular content projection, which component's context do projected bindings resolve against — the parent or the child? Why does this matter?"

Strong answer covers: - Projected content bindings resolve against the parent component's context - The rule: "compiled in the parent, rendered in the child" - `{{ variable }}` inside projected content refers to the parent's variable property - The child cannot directly expose its data to projected content - When child-to-content data flow is needed, `TemplateRef` + `ngTemplateOutlet` is the correct pattern

Weak answers miss: The architectural implication — that `ng-content` creates a passive child (display contract), while `TemplateRef` creates an active child (rendering contract).

QUESTION 3 — Senior "If you put `@if (isOpen)` around `<ng-content>` in a child component, does that prevent the projected content from initialising until `isOpen` is true?"

Strong answer covers: - No — projected content is part of the parent's template and instantiates with the parent - The child's `@if` only controls display — the content is already created before the `@if` evaluates - `ngOnInit` in a projected component fires immediately on parent mount, regardless of the child's conditions - This matters for performance: heavy components behind an `@if (false)` slot still initialise - The fix: use `TemplateRef` input — pass a blueprint, not live content — so the child controls instantiation

Weak answers miss: The root cause — that projected content is owned by the parent's template, not the child's.

Section 15 — Anti-Patterns to Avoid in Production

ANTI-PATTERN 1 — Deep Component Trees Communicating Through Projected Content

```
// ngLayout has currentUser, but projected content can't access it
@Component({
  selector: 'app-layout',
  template: `<div class="layout"><ng-content></ng-content></div>`
})
export class LayoutComponent {
  currentUser = inject(UserService).user;
}

// Parent must duplicate the injection
@Component({
  template: `
    <app-layout>
      <app-profile [user]="currentUser"></app-profile>
    </app-layout>
  `
})
export class ParentComponent {
  // Developer forced to duplicate currentUser injection here
  currentUser = inject(UserService).user;
}
```

Why it seems fine: The layout component "has" the user — it seems natural that projected content inside it would have access too.

What breaks in production: Every component that uses `<app-layout>` must independently inject `UserService`. Duplicated injection and confusion about data ownership.

Correct pattern: Use a shared service (Topic 27), signals, or switch to `TemplateRef` with context.

ANTI-PATTERN 2 — Styling Projected Content with Child-Scoped Styles

```
/* In child component's CSS — won't apply to projected content */
p { color: #374151; line-height: 1.6; }
h3 { font-size: 1.25rem; font-weight: 600; }
```

Why it seems fine: The styles are in the child component's CSS file, and the projected content renders inside the child's template.

What breaks in production: View encapsulation scopes Angular component styles to that component's own template elements. Projected content carries the parent's style scope.

Correct pattern:

```
/* Style the WRAPPER – inherited CSS cascades into projected content */
.card-body { color: #374151; line-height: 1.6; font-size: 1rem; }
```

ANTI-PATTERN 3 — Wrapping ng-content in Unnecessary Elements

```
@Component({
  selector: 'app-scrollable',
  template: `
    <div class="scroll-wrapper" cdkScrollable>
      <div class="inner-wrapper">
        <div class="content-wrapper">
          <ng-content></ng-content>
        </div>
      </div>
    </div>
  `
})
```

Why it seems fine: Each wrapper has a reason. The developer adds them iteratively as styling needs arise.

What breaks in production: Three wrapper divs break CSS layout — especially flexbox and grid children, position: sticky within scroll containers, and form element focus management.

Correct pattern:

```
@Component({
  selector: 'app-scrollable',
  styles: [`:host { display: block; overflow-y: auto; max-height: 400px; padding: 16px; }`],
  template: `<ng-content></ng-content>`
})
// Use :host CSS pseudo-selector to style the component's own element.
```

Section 16 — Mental Model Summary

Core concept: ng-content is a slot placeholder in a child component's template — it marks where the parent's projected HTML will be inserted at runtime. The parent writes the content; the child provides the frame.

Key rules: 1. Projected content bindings always resolve against the parent — {{ x }} in projected content reads the parent's x 2. Projected content instantiates with the parent — @if around <ng-content> only hides, never delays init 3. select="[attr]" routes specific parent content to specific slots using CSS selectors 4. Content matching no selector is silently discarded unless a default <ng-content> (no select) exists 5. ng-content can only project content once — for repeated stamping with data, use TemplateRef + ngTemplateOutlet 6. Child component CSS

does not style projected content — use wrapper inheritance or global styles

The single most important Angular rule: Projected content always compiles in the parent's context — the child component cannot access or modify its projected content's bindings.

Content Projection vs TemplateRef — When to Use Which:

Criteria	ng-content	TemplateRef + ngTemplateOutlet
-----	-----	-----
Parent provides	Live HTML / components	Unrendered blueprint
Child can pass data in	No	Yes (via context)
Lazy instantiation	No (always with parent)	Yes (child controls when)
Same template multiple	Once only	Unlimited
Complexity	Low	Medium
Best for	Modals, cards, panels	Data grids, virtual lists, lazy tabs

TOPIC 32 — ViewChild, ViewChildren, ContentChild, ContentChildren

Angular Core — Components & Architecture

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The decorator syntax is simple, but understanding when references are available (which lifecycle hook), the difference between the four decorators, and using `QueryList` correctly requires solid understanding of Angular's rendering pipeline and the View vs Content distinction from Topic 31.

Angular Relevance: High — These decorators are used in every non-trivial Angular component that needs programmatic access to child elements, child components, or projected content. Form components, animation triggers, third-party library integration, and interactive containers all depend on them.

Section 1 — Explanation

THE THEATRE DIRECTOR ANALOGY

`@ViewChild` is the director having a direct line to one specific actor in their own production — an actor the director personally cast and placed on stage. "Get me the lead actor" — they hand back a direct reference. The director can tell that actor to do anything: change their position, speak louder, exit stage left.

`@ViewChildren` is the same director but asking for all actors of a certain type — "get me every actor playing a guard." They hand back a roster. The director can loop through the roster and give each one instructions.

`@ContentChild` is the director reaching into the guest actor area — actors that a guest director (the parent component) brought in and placed on the stage via content projection. The host director can reference one of those guest actors, but didn't cast them and doesn't fully control them.

`@ContentChildren` is the same but for all guest actors of a given type.

The crucial timing rule: your own actors (`@View*`) are ready after `ngAfterViewInit`. Guest actors (`@Content*`) are ready after `ngAfterContentInit`.

TECHNICAL EXPLANATION

All four decorators are template queries — they search the component's template (or projected content) for matching elements or components and give you a live TypeScript reference to them.

The View vs Content distinction (building on Topic 31): - View = elements defined in the component's own template - Content = elements projected into the component via `ng-content`

The Single vs Multiple distinction: - Child = returns one result (the first match) as a direct reference - Children = returns all matches as a QueryList<T> (a live, iterable collection)

Quick Reference Table:

Decorator	Searches	Returns	Available From
-----	-----	-----	-----
@ViewChild	Own template	T undefined	ngAfterViewInit
@ViewChildren	Own template	QueryList<T>	ngAfterViewInit
@ContentChild	Projected content	T undefined	ngAfterContentInit
@ContentChildren	Projected content	QueryList<T>	ngAfterContentInit

What you can query: - A component type — gets a reference to the child component instance - A directive type — gets a reference to the directive instance - A template reference variable (#myRef) — gets the ElementRef or TemplateRef - An ElementRef — gets the raw DOM element wrapper - A TemplateRef — gets a template blueprint

The { static: true/false } option: - static: false (default) — query resolved after change detection runs; required for queries inside @if/@for - static: true — query resolved at component init time, before change detection; only use when the queried element is always present (never inside @if or @for)

Section 2 — Connection to Prior Concepts

Direct dependencies: - Topic 25 — Component Lifecycle Hooks: the entire API depends on knowing which hook gives you access — ngAfterViewInit for @ViewChild/@ViewChildren, ngAfterContentInit for @ContentChild/@ContentChildren - Topic 31 — Content Projection: @ContentChild/@ContentChildren are meaningless without understanding what content projection is - Topic 26 — @Input() & @Output(): once you have a @ViewChild reference to a child component, you can call its public methods - Topic 15 — Access Modifiers: @ViewChild gives you the component instance — only public methods and properties are accessible from outside - Topic 23 — Angular Signals: QueryList has a changes Observable; combining this with signals is a modern pattern - Topic 30 — ngTemplateOutlet: @ViewChild is often used to get a TemplateRef reference to pass to ngTemplateOutlet programmatically

Bridging note: In Topic 31 you built a tab component that used @ContentChildren to read projected TabPanelComponent instances. That pattern — a container component querying its projected children to drive its own behaviour — is the canonical use case for @ContentChildren. This topic explains exactly how that query works, when the results are available, and how to react when the projected content changes.

Most directly extends: Topic 31 — Content Projection and Topic 25 — Component Lifecycle Hooks

Section 3 — Code Example

PLAIN JS — THE CORE CONCEPT IN ISOLATION

```

// @ViewChild equivalent – get a reference to a specific child element
const inputEl = document.querySelector('#myInput');
inputEl.focus();

// @ViewChildren equivalent – get all matching children
const allButtons = document.querySelectorAll('.action-btn');
allButtons.forEach(btn => btn.setAttribute('disabled', 'true'));

// The problems Angular's decorators solve:
// 1. querySelector returns null until the element exists in the DOM
//    Angular's ngAfterViewInit guarantees the element exists
// 2. querySelector searches the entire document
//    Angular's queries are scoped to THIS component's template only
// 3. querySelector returns a static snapshot
//    Angular's QueryList is LIVE – it updates when @for adds/removes elements
// 4. querySelector returns HTMLElement – no type safety
//    @ViewChild returns the actual typed component/directive instance

```

REAL ANGULAR EXAMPLE — @ViewChild WITH TEMPLATE REFERENCE AND COMPONENT

```

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <div class="counter">
      <button (click)="decrement()">-</button>
      <span>{{ count() }}</span>
      <button (click)="increment()">+</button>
    </div>
  `
})
export class CounterComponent {
  count = signal(0);
  increment(): void { this.count.update(n => n + 1); }
  decrement(): void { this.count.update(n => n - 1); }
  reset(): void { this.count.set(0); }
  getValue(): number { return this.count(); }
}

@Component({
  selector: 'app-game',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <input #scoreInput type="number" placeholder="Set score">
    <app-counter #scoreCounter></app-counter>
    <ng-template #winnerBanner>
      <div class="winner">You Win!</div>
    </ng-template>
    <div class="controls">
      <button (click)="focusInput()">Focus Input</button>
      <button (click)="resetScore()">Reset Score</button>
      <button (click)="logScore()">Log Score</button>
    </div>
    @if (showWinner()) {
      <ng-container *ngTemplateOutlet="winnerBannerRef!"></ng-container>
    }
  `
})

```

```

    })
    export class GameComponent implements AfterViewInit {
        showWinner = signal(false);

        // @ViewChild by template reference variable (#scoreInput)
        // static: true – the input is always in the template, never inside @if
        @ViewChild('scoreInput', { static: true })
        scoreInputRef!: ElementRef<HTMLInputElement>;

        // @ViewChild by component type – returns the CounterComponent instance
        @ViewChild(CounterComponent)
        counterComponent?: CounterComponent;

        // @ViewChild for a TemplateRef
        @ViewChild('winnerBanner')
        winnerBannerRef?: any;

        ngAfterViewInit(): void {
            // SAFE: both refs are guaranteed to be set by this hook
            console.log('Input element:', this.scoreInputRef.nativeElement);
            console.log('Counter value:', this.counterComponent?.getValue());
            // DO NOT access @ViewChild refs in ngOnInit – they're not ready yet
        }

        focusInput(): void {
            this.scoreInputRef.nativeElement.focus();
            this.scoreInputRef.nativeElement.select();
        }

        resetScore(): void {
            this.counterComponent?.reset();
        }

        logScore(): void {
            const score = this.counterComponent?.getValue();
            console.log('Current score:', score);
            if (score && score >= 10) {
                this.showWinner.set(true);
            }
        }
    }
}

```

@ViewChildren WITH QueryList

```

@Component({
    selector: 'app-card-grid',
    standalone: true,
    imports: [CardComponent],
    template: `
        <div class="grid">
            @for (card of cards(); track card.id) {
                <app-card [title]="card.title"></app-card>
            }
        </div>
        <div class="controls">
            <button (click)="highlightAll()">Highlight All</button>
            <button (click)="clearAll()">Clear All</button>
        </div>
    `
})

```



```

        <button (click)="highlightFirst()">Highlight First</button>
    </div>
    ,
  })
  export class CardGridComponent implements AfterViewInit {
    cards = signal([
      { id: 1, title: 'Card One' },
      { id: 2, title: 'Card Two' },
      { id: 3, title: 'Card Three' },
    ]);

    // @ViewChildren by component type – returns QueryList<CardComponent>
    @ViewChildren(CardComponent)
    cardComponents!: QueryList<CardComponent>;

    ngAfterViewInit(): void {
      console.log('Number of cards:', this.cardComponents.length);
      console.log('First card:', this.cardComponents.first);
      console.log('Last card:', this.cardComponents.last);

      // React to QueryList changes (e.g. when @for adds/removes items)
      this.cardComponents.changes.subscribe((updatedList: QueryList<CardComponent>) => {
        console.log('Cards changed – new count:', updatedList.length);
      });
    }

    highlightAll(): void {
      this.cardComponents.forEach(card => card.highlight());
    }

    clearAll(): void {
      this.cardComponents.forEach(card => card.clearHighlight());
    }

    highlightFirst(): void {
      this.cardComponents.first?.highlight();
    }
  }
}

```

@ContentChild AND @ContentChildren

```

@Component({
  selector: 'app-tab-group',
  standalone: true,
  template: `
    <div class="tab-bar">
      @for (tab of tabs; track tab.tabId) {
        <button
          [class.active]="activeTabId() === tab.tabId"
          (click)="selectTab(tab.tabId)"
        >
          {{ tab.label }}
        </button>
      }
    </div>
    <div class="tab-content">
      <ng-content></ng-content>
    </div>
  `
})

```

```

    })
    export class TabGroupComponent implements AfterContentInit {
        activeTabId = signal('');

        // @ContentChildren – queries PROJECTED app-tab components
        // These are NOT in this component's own template – they're projected in
        @ContentChildren(TabComponent)
        tabs!: QueryList<TabComponent>;

        // @ContentChild – queries a single projected element by template ref
        @ContentChild('groupHeader')
        headerTemplate?: any;

        ngAfterContentInit(): void {
            // SAFE: projected content is available from this hook
            if (this.tabs.length > 0) {
                const firstTab = this.tabs.first;
                this.activeTabId.set(firstTab.tabId);
                firstTab.activate();
            }

            // React to changes in projected content
            this.tabs.changes.subscribe(() => {
                console.log('Tabs changed – new count:', this.tabs.length);
            });
        }

        selectTab(tabId: string): void {
            this.tabs.forEach(tab => tab.deactivate());
            const selected = this.tabs.find(tab => tab.tabId === tabId);
            if (selected) {
                selected.activate();
                this.activeTabId.set(tabId);
            }
        }
    }
}

```

Section 4 — Before & After Refactor

BEFORE — DIRECT DOM MANIPULATION INSTEAD OF @ViewChild

```

@Component({
    selector: 'app-login',
    template: `
        <input id="emailField" type="email" placeholder="Email">
        <button (click)="focusEmail()">Focus Email</button>
    `
})
export class LoginComponent implements OnInit {
    ngOnInit(): void {
        // Problem 1: might run before the DOM is ready
        // Problem 2: searches the ENTIRE document, not just this component
        // Problem 3: breaks with SSR – document doesn't exist server-side
        // Problem 4: ID might clash with another component
        const input = document.getElementById('emailField') as HTMLInputElement;
    }
}

```

```

    input?.focus();
  }

  focusEmail(): void {
    (document.getElementById('emailField') as HTMLInputElement)?.focus();
  }
}

```

What breaks: - `document.getElementById` searches the entire document - Running in `ngOnInit` is too early — the DOM may not be rendered yet - Breaks with Angular Universal (SSR) — document doesn't exist on the server - Bypasses Angular's change detection

AFTER — `@ViewChild` WITH `ElementRef`

```

@Component({
  selector: 'app-login',
  standalone: true,
  template: `
    <!-- Template reference variable – no ID needed -->
    <input #emailField type="email" placeholder="Email">
    <button (click)="focusEmail()">Focus Email</button>
  `
})
export class LoginComponent implements AfterViewInit {
  // Scoped to THIS component's template only
  // Type-safe – ElementRef<HTMLInputElement>
  // Available at the right time (AfterViewInit)
  @ViewChild('emailField')
  emailInput!: ElementRef<HTMLInputElement>;

  ngAfterViewInit(): void {
    // Guaranteed to be ready here
    this.emailInput.nativeElement.focus();
  }

  focusEmail(): void {
    this.emailInput.nativeElement.focus();
  }
}

```

Why this matters in Angular: `@ViewChild` is scoped to the component, SSR-compatible, type-safe, and timed correctly. It's the Angular-idiomatic way to access DOM elements and child component instances.

Section 5 — Common Mistakes & Misconceptions

MISTAKE 1 — Accessing `@ViewChild` References in `ngOnInit`

What the mistake looks like:

```

@ViewChild('myInput') inputRef!: ElementRef;

ngOnInit(): void {
  // inputRef is undefined here – the view hasn't been created yet
  this.inputRef.nativeElement.focus(); // TypeError
}

```

```
}
```

Why developers make it: `ngOnInit` feels like "component is ready" — but the view (the template DOM) isn't built until after `ngOnInit` completes.

How to avoid it: Use `ngAfterViewInit` for `@ViewChild/@ViewChildren`. Use `ngAfterContentInit` for `@ContentChild/@ContentChildren`. Mnemonic: the hook name tells you when it's ready — `AfterViewInit` -> `@ViewChild`. `AfterContentInit` -> `@ContentChild`.

Angular-specific consequence: `TypeError` at runtime: Cannot read properties of undefined. With `static: true`, the ref is available in `ngOnInit` — but only when the element is never inside `@if/@for`.

MISTAKE 2 — Using `static: true` on Elements Inside `@if` or `@for`

What the mistake looks like:

```
// Template: @if (showForm) { <input #formInput> }
@ViewChild('formInput', { static: true }) // static: true on conditional element
formInput!: ElementRef;

ngOnInit(): void {
  // formInput is undefined even with static: true
  this.formInput.nativeElement.focus(); // TypeError
}
```

Why developers make it: `static: true` sounds like "always available."

How to avoid it: `static: true` means "resolve before change detection runs" — it only works for elements that are unconditionally present in the template at render time. If the element is inside `@if`, `@for`, or `@switch`, use `static: false` (default) and access in `ngAfterViewInit`.

Angular-specific consequence: undefined at runtime with a confusing error. If `showForm` starts true, the reference is set. If it starts false, it's undefined — intermittent bugs depending on initial state.

MISTAKE 3 — Modifying `@ViewChild` References Inside `ngAfterViewInit` Without Correct Timing

What the mistake looks like:

```
@ViewChild(ChildComponent) child!: ChildComponent;

ngAfterViewInit(): void {
  // This triggers a change detection cycle during change detection
  // Angular throws: ExpressionChangedAfterItHasBeenCheckedError
  this.someProperty = this.child.getValue();
}
```

Why developers make it: `ngAfterViewInit` seems like the natural place to read child values and update parent state.

How to avoid it:

```
ngAfterViewInit(): void {
  // Option 1: Use setTimeout to defer to next change detection cycle
  setTimeout(() => {
    this.someProperty = this.child.getValue();
  });
}
```

```

// Option 2: Use a Signal – signal updates don't trigger this error
this.someSignal.set(this.child.getValue());

// Option 3: Use ChangeDetectorRef.detectChanges() explicitly
this.someProperty = this.child.getValue();
this.cdr.detectChanges();
}

```

Angular-specific consequence: ExpressionChangedAfterItHasBeenCheckedError in development mode. In production mode, the UI shows the old value, causing subtle display bugs.

Section 6 — Do's and Don'ts

DO: - Access `@ViewChild` / `@ViewChildren` in `ngAfterViewInit` - Access `@ContentChild` / `@ContentChildren` in `ngAfterContentInit` - Use `static: true` only for elements unconditionally present in the template - Use optional chaining (`this.child?.method()`) since `@ViewChild` can be undefined - Call public methods and read public properties via `@ViewChild` - Subscribe to `queryList.changes` to react to dynamic list updates

DON'T: - Access them in `ngOnInit` — the view isn't built yet - Access content queries in `ngAfterViewInit` — use the matching lifecycle hook - Use `static: true` for elements inside `@if`, `@for`, or `@switch` - Use the non-null assertion (`this.child!.method()`) without a runtime check - Reach into private properties of a child component — respect encapsulation - Read `queryList.length` in `ngAfterViewInit` and assume it won't change — it can

Section 7 — Debugging Tips

BUG 1 — `@ViewChild` Reference Is undefined at Runtime

Symptom: `TypeError: Cannot read properties of undefined (reading 'nativeElement')`. Occurs when trying to use the `@ViewChild` reference.

Cause: One of three things — (a) accessed in the wrong lifecycle hook (`ngOnInit` instead of `ngAfterViewInit`), (b) element is inside `@if (false)` when you try to access it, or (c) `static: true` on a conditional element.

Fix:

```

// Check 1: Are you in the right hook?
ngAfterViewInit(): void {
  console.log('ViewChild ref:', this.myRef); // Should log the reference
}

// Check 2: Is the element conditionally rendered?
// @if (showPanel) { <app-panel #panel> }
// myRef is undefined when showPanel = false – guard with optional chaining:
this.panelRef?.someMethod();

// Check 3: Is static: true set on a conditional element? Remove it.
@ViewChild('panel') panelRef?: ChildComponent; // static: false (default)

```

Angular-specific note: Add `console.log(this.myRef)` in `ngAfterViewInit` as a first step.

BUG 2 — QueryList Shows Correct Length But Iterating Gives Wrong Results

Symptom: `this.items.length` is correct, but calling `forEach()` or `toArray()` gives stale results after a list update.

Cause: You're reading the `QueryList` synchronously after a signal/data update but before Angular has run change detection and re-rendered the `@for` loop.

Fix:

```
// Reading QueryList immediately after data change (stale)
this.myData.update(list => [...list, newItem]);
console.log(this.viewChildren.length); // Still shows old count

// Subscribe to changes – fires after DOM updates
this.viewChildren.changes.subscribe((list: QueryList<MyComponent>) => {
  console.log('Updated count:', list.length); // Correct
});
```

Angular-specific note: `QueryList.changes` is an `Observable` that emits after every change detection cycle where the list has changed.

BUG 3 — ExpressionChangedAfterItHasBeenCheckedError After Setting Property in ngAfterViewInit

Symptom: Console error in development mode when you set a component property inside `ngAfterViewInit`.

Cause: Angular runs change detection, then calls `ngAfterViewInit`, then runs a second check (dev mode only) to verify nothing changed. If you set a property that affects the template, the second check sees a different value and throws.

Fix:

```
// Use setTimeout – defers to next event loop tick
ngAfterViewInit(): void {
  setTimeout(() => {
    this.title = this.childComponent.getTitle();
  });
}

// Or use Signals – signal.set() schedules correctly
ngAfterViewInit(): void {
  this.titleSignal.set(this.childComponent.getTitle());
}
```

Angular-specific note: This error only appears in development mode. Don't suppress it — fix the root cause.

Section 8 — Real-World Applications

SCENARIO 1 — Auto-Focus Input on Modal Open

```

@Component({
  selector: 'app-edit-modal',
  standalone: true,
  template: `
    <div class="modal" role="dialog">
      <h2>{{ title }}</h2>
      <input #firstInput [(ngModel)]="value" [placeholder]="placeholder" type="text">
      <div class="actions">
        <button (click)="cancel.emit()">Cancel</button>
        <button (click)="confirm.emit(value)">Save</button>
      </div>
    </div>
  `
})
export class EditModalComponent implements AfterViewInit, OnChanges {
  @Input() title = '';
  @Input() placeholder = '';
  @Input() initialValue = '';
  @Input() isOpen = false;
  @Output() confirm = new EventEmitter<string>();
  @Output() cancel = new EventEmitter<void>();
  value = '';

  @ViewChild('firstInput')
  firstInputRef?: ElementRef<HTMLInputElement>;

  ngAfterViewInit(): void {
    if (this.isOpen) {
      this.focusInput();
    }
  }

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['isOpen']?.currentValue === true) {
      setTimeout(() => this.focusInput());
    }
    if (changes['initialValue']) {
      this.value = this.initialValue;
    }
  }

  private focusInput(): void {
    this.firstInputRef?.nativeElement.focus();
    this.firstInputRef?.nativeElement.select();
  }
}

```

SCENARIO 2 — Accordion Component Using @ContentChildren

```

@Component({
  selector: 'app-accordion-item',
  standalone: true,
  template: `
    <div class="accordion-item" [class.open]="isOpen()">
      <button class="accordion-header" (click)="toggle()">
        <span>{{ title }}</span>
        <span class="chevron">{{ isOpen() ? 'v' : '>' }}</span>
      </button>
    </div>
  `
})

```

```

        </button>
        <div class="accordion-body" [hidden]="!isOpen()">
            <ng-content></ng-content>
        </div>
    </div>
    ,
})
export class AccordionItemComponent {
    @Input() title = '';
    isOpen = signal(false);
    open(): void { this.isOpen.set(true); }
    close(): void { this.isOpen.set(false); }
    toggle(): void { this.isOpen.update(v => !v); }
}

@Component({
    selector: 'app-accordion',
    standalone: true,
    template: `
        <div class="accordion">
            <ng-content></ng-content>
        </div>
    `
})
export class AccordionComponent implements AfterContentInit {
    @Input() exclusive = true;

    @ContentChildren(AccordionItemComponent)
    items!: QueryList<AccordionItemComponent>;

    ngAfterContentInit(): void {
        this.items.first?.open();
        this.items.changes.subscribe(() => {
            console.log('Accordion items changed:', this.items.length);
        });
        if (this.exclusive) {
            this.setupExclusiveMode();
        }
    }

    private setupExclusiveMode(): void {
        this.items.forEach(item => {
            const originalOpen = item.open.bind(item);
            item.open = () => {
                this.closeAll();
                originalOpen();
            };
        });
    }

    closeAll(): void { this.items.forEach(item => item.close()); }
    openAll(): void {
        if (!this.exclusive) { this.items.forEach(item => item.open()); }
    }
}

// PARENT USAGE
@Component({

```



```

template: `
  <app-accordion [exclusive]="true">
    <app-accordion-item title="What is Angular?">
      <p>Angular is a TypeScript-based web framework by Google.</p>
    </app-accordion-item>
    <app-accordion-item title="What are Signals?">
      <p>Signals are Angular's reactive primitive – introduced in v16.</p>
    </app-accordion-item>
    <app-accordion-item title="What is the Angular CLI?">
      <p>The Angular CLI automates project setup, building, and testing.</p>
    </app-accordion-item>
  </app-accordion>
`
  })
  export class FaqPageComponent {}

```

Section 9 — Practice Exercises

EXERCISE 1 — Beginner Build a standalone `FocusableInput` component with a text input inside. Use `@ViewChild` with a template reference variable to get the `ElementRef` of the input. Expose a public `focus()` method on `FocusableInput` that calls `.focus()` on the native element. In a parent component, use `@ViewChild` to get the `FocusableInput` instance and call its `focus()` method when a "Focus Field" button is clicked. Verify the input receives focus correctly.

EXERCISE 2 — Intermediate Build a `Carousel` component that contains a dynamic list of `CarouselSlide` components (each with a title and content input). Use `@ViewChildren(CarouselSlide)` to get references to all slides. Implement `next()` and `prev()` methods on the `Carousel` that programmatically call `activate()` and `deactivate()` methods on the individual slide instances to show/hide them. Display the current slide index using interpolation. Add keyboard navigation by listening to (`keydown`) on the carousel wrapper element, accessed via `@ViewChild`.

EXERCISE 3 — Advanced Build a `FormSection` component and a `FormWizard` container component. Each `FormSection` has an `@Input()` `sectionTitle`, a `validate(): boolean` method that checks if its projected form fields are filled, and a `markComplete()` method. The `FormWizard` uses `@ContentChildren(FormSection)` to query all projected sections. Implement: (a) a step indicator that renders a step for each projected section using the queried list, (b) "Next" and "Back" navigation that calls `validate()` on the current section before advancing, and (c) a `markComplete()` call on successfully validated sections. React to `QueryList.changes` to handle sections being dynamically added or removed.

Section 10 — Key Takeaways

Core Concepts: - `@ViewChild` — queries one element/component/directive from the component's own template; returns the instance or undefined - `@ViewChildren` — queries all matching elements from the component's own template; returns a live `QueryList<T>` - `@ContentChild` — queries one element/component from projected content; returns the instance or undefined - `@ContentChildren` — queries all matching from projected content; returns a live

QueryList<T> - QueryList<T> — a live, iterable collection; updates automatically when @for adds/removes items; has .first, .last, .forEach(), .toArray(), .changes (Observable) - { static: true } — resolves query before change detection; only for elements unconditionally in the template; enables access in ngOnInit - { static: false } — (default) resolves after change detection; required for elements inside @if/@for; access in ngAfterViewInit/ngAfterContentInit

Quick Reference:

Decorator	Source	Returns	Hook	static
@ViewChild	Own template	T undefined	ngAfterViewInit	Optional
@ViewChildren	Own template	QueryList<T>	ngAfterViewInit	N/A
@ContentChild	Projected content	T undefined	ngAfterContentInit	Optional
@ContentChildren	Projected content	QueryList<T>	ngAfterContentInit	N/A

Angular-Specific Rules: - Never access query results in ngOnInit — the view/content isn't built yet (unless static: true on an unconditional element) - @ViewChild gives access to the full component instance — only call public methods - Setting template-bound properties in ngAfterViewInit causes ExpressionChangedAfterItHasBeenCheckedError — use setTimeout, signals, or detectChanges() - Subscribe to queryList.changes for reactive responses to dynamic list updates

One-Line Memory Aid: View queries (@ViewChild) read your own template after ngAfterViewInit; Content queries (@ContentChild) read projected content after ngAfterContentInit — the hook name always tells you when it's ready.

Section 11 — Thought-Provoking Question + Answer

Question: @ViewChild gives you a direct reference to a child component instance, letting you call its methods and read its properties directly from the parent. But Angular's standard data flow uses @Input()/@Output() for parent-child communication. If @ViewChild can do everything @Input()/@Output() can and more, why isn't it the default pattern? When does reaching for @ViewChild signal a design problem?

Answer:

@ViewChild is more powerful but less predictable than @Input()/@Output(). The distinction matters enormously at scale.

Conceptual explanation: @Input()/@Output() creates an explicit, declared contract between parent and child. The child says "I accept these inputs, I emit these outputs." The parent says "I provide these values, I handle these events." Both sides are visible, testable, and documented in the component's public API. Change detection knows exactly what to watch.

@ViewChild creates an implicit, imperative dependency. The parent reaches into the child and calls methods or reads state directly. The child doesn't know it's being called — it has no say in when or how. The parent can bypass the child's intended API entirely.

Why this causes problems at scale:

Testability: Testing a component that uses `@Input()/@Output()` is straightforward. Testing a component that uses `@ViewChild` to imperatively control a child requires mocking or providing the full child component tree.

Change detection predictability: Angular's `OnPush` change detection relies on the input reference change pattern. When a parent calls `child.method()` via `@ViewChild`, `OnPush` components don't automatically know to re-check.

Coupling: `@ViewChild` couples the parent to the child's implementation, not just its interface. If the child renames a method or refactors internally, the parent breaks silently.

When `@ViewChild` is the right tool: DOM interactions that have no input/output equivalent — `focus()`, `scrollIntoView()`, `play()` on a video element. Integrating third-party components that expose imperative APIs. Reading geometry (`getBoundingClientRect`) for animation calculations.

Practical rule: Use `@Input()/@Output()` for data and events. Reach for `@ViewChild` only for imperative DOM or lifecycle operations that cannot be expressed as data flow. If you find yourself using `@ViewChild` to read a child's state and update the parent — that state should probably live in a shared service or be lifted up as an `@Output()`.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts: - Iterators and iterables — `QueryList` implements the iterable protocol; understanding iterators explains why `for...of` works on a `QueryList` - `WeakRef` / object references — understanding that `@ViewChild` gives you a live object reference (not a snapshot)

TypeScript Concepts: - Generic types — `QueryList<T>` and `@ViewChild` return typed references; understanding generics (Topic 16) enables correct typing - Intersection types — sometimes `@ViewChild` returns a type that combines `ElementRef` and a component reference

Angular-Specific Concepts: - Topic 33 — Reactive Forms — `@ViewChild` is used to access form components - Topic 41 — Custom Directives — `@ViewChild` can query directives, not just components - Topic 43 — Angular Animations — `@ViewChild` is used to trigger imperative animations on child elements - Topic 47 — Testing Angular Components — testing components that use `@ViewChild` and `@ContentChildren` requires specific `TestBed` setup

Section 13 — Study Plan Checkpoint

This topic directly depends on: - Topic 15 — Access Modifiers (only public members are accessible via `@ViewChild`) - Topic 23 — Angular Signals (`QueryList.changes` + signals; signal updates in `ngAfterViewInit`) - Topic 25 — Component Lifecycle Hooks (the entire API is lifecycle-hook-dependent) - Topic 26 — `@Input()` & `@Output()` (the contrast pattern; knowing when to use each) - Topic 31 — Content Projection (`@ContentChild/@ContentChildren` are meaningless without it)

This topic unlocks: - Topic 33 — Reactive Forms — `@ViewChild` is used to imperatively access form groups and trigger validation - Topic 41 — Custom Directives — querying directives via

@ViewChild is a key integration pattern - Topic 43 — Angular Animations — imperative animation triggers use @ViewChild to access elements - Topic 47 — Testing Angular Components — testing @ViewChild/@ContentChildren requires TestBed with proper content projection setup

Production readiness assessment: You are ready to use @ViewChild for DOM manipulation (focus, scroll) and @ContentChildren for container components (tabs, accordions) confidently in production. Before using @ViewChild to call child component methods extensively, apply the design principle from Section 11 — if you're reading child state to update parent state, that's a signal to refactor toward @Output() or a shared service instead.

Section 14 — Common Interview Questions

QUESTION 1 — Mid "What is the difference between @ViewChild and @ContentChild? When would you use each?"

Strong answer covers: - @ViewChild queries elements defined in the component's own template - @ContentChild queries elements projected into the component via ng-content (the parent's content) - @ViewChild available in ngAfterViewInit; @ContentChild in ngAfterContentInit - Real example: a tab container uses @ContentChildren(TabComponent) to query projected tabs - @ViewChild example: auto-focusing an <input> defined in the component's own template

Weak answers miss: The lifecycle hook difference and the real-world use case that makes @ContentChildren necessary.

QUESTION 2 — Mid "What is QueryList and why is it described as 'live'? How do you react to changes in a QueryList?"

Strong answer covers: - QueryList<T> is Angular's collection type returned by @ViewChildren and @ContentChildren - "Live" means it automatically updates when @for adds/removes items from the DOM - It's iterable — supports forEach, toArray(), first, last, length - queryList.changes is an Observable that emits the updated QueryList after each change detection cycle where the list changed - Subscribe to .changes in ngAfterViewInit/ngAfterContentInit to react to dynamic list changes - Don't cache queryList.length — read it fresh each time or subscribe to .changes

Weak answers miss: The .changes Observable and that reading queryList.length outside a .changes subscription can give stale results.

QUESTION 3 — Mid-Senior "When should you use { static: true } on a @ViewChild query, and what breaks if you use it incorrectly?"

Strong answer covers: - static: true resolves the query at component initialisation time, before change detection runs — making the reference available in ngOnInit - Only safe when the element is unconditionally present in the template (never inside @if, @for, @switch) - static: false (default) resolves after change detection — required for conditional elements - Using static: true on a conditional element gives undefined - If the element starts visible (condition is initially true), the reference is set — but breaks intermittently when the condition starts false - Rule of thumb: use static: false (default) unless you specifically need the reference in ngOnInit

Weak answers miss: The intermittent failure mode — that static: true on a conditional element doesn't always fail; it depends on the initial condition value.

Section 15 — Anti-Patterns to Avoid in Production

ANTI-PATTERN 1 — Using @ViewChild as a State Synchronisation Mechanism

```
@Component({
  template: `
    <app-counter #counter></app-counter>
    <p>Total: {{ getTotal() }}</p>  <!-- calls @ViewChild method on every CD cycle -->
  `
})
export class ParentComponent {
  @ViewChild('counter') counter?: CounterComponent;

  // Called on every change detection cycle – reads child state
  getTotal(): number {
    return this.counter?.getValue() ?? 0;
  }
}
```

Why it seems fine: It works. The parent can always read the latest child value.

What breaks in production: getTotal() calls counter.getValue() on every change detection cycle. State is duplicated between parent and child with no single source of truth. Testing requires the full child component. OnPush breaks silently.

Correct pattern:

```
// Child emits via @Output()
// <app-counter (valueChange)="displayTotal = $event"></app-counter>
// <p>Total: {{ displayTotal }}</p>
```

ANTI-PATTERN 2 — Calling @ContentChildren Results Before ngAfterContentInit

```
@Component({...})
export class TabsComponent implements OnInit, AfterContentInit {
  @ContentChildren(TabComponent) tabs!: QueryList<TabComponent>;

  ngOnInit(): void {
    // tabs is an empty QueryList here – content isn't projected yet
    console.log('Tab count in OnInit:', this.tabs.length); // 0
    this.tabs.first?.activate(); // Error or silent no-op
  }

  ngAfterContentInit(): void {
    // tabs is populated here – projected content is fully available
    this.tabs.first?.activate(); // Works correctly
  }
}
```

Why it seems fine: ngOnInit runs at component initialisation. And the error is subtle: tabs.length is 0, not an exception.

What breaks in production: Default tab is never activated. The component appears to work in some scenarios but never shows the correct initial state.

Correct pattern: Always access @ContentChildren results in ngAfterContentInit. No exceptions.

ANTI-PATTERN 3 — Storing QueryList Results in a Local Array Without Subscribing to Changes

```
export class MenuComponent implements AfterViewInit {
  @ViewChildren(MenuItemComponent) menuItems!: QueryList<MenuItemComponent>;
  private itemsArray: MenuItemComponent[] = [];

  ngAfterViewInit(): void {
    // Takes a one-time snapshot – never updated when items change
    this.itemsArray = this.menuItems.toArray();
  }

  activateItem(index: number): void {
    // itemsArray is stale if items were added/removed after init
    this.itemsArray[index]?.activate();
  }
}
```

Why it seems fine: toArray() converts the live QueryList to a regular array — convenient for use with array methods. In static apps it never goes stale.

What breaks in production: When menu items are dynamically added/removed, itemsArray is stale. activateItem(2) activates the wrong item or throws because the index no longer matches.

Correct pattern:

```
ngAfterViewInit(): void {
  // Subscribe to changes to react reactively
  this.menuItems.changes.subscribe(() => {
    console.log('Menu updated:', this.menuItems.length);
  });
}

activateItem(index: number): void {
  // Read fresh from QueryList every time – never stale
  this.menuItems.toArray()[index]?.activate();
}
```

Section 16 — Mental Model Summary

Core concept: @ViewChild/@ViewChildren give you typed TypeScript references to elements and components in your own template. @ContentChild/@ContentChildren give you the same for projected content. All four are template queries — they search the DOM and return live references.

Key rules: 1. @ViewChild / @ViewChildren -> access in ngAfterViewInit 2. @ContentChild / @ContentChildren -> access in ngAfterContentInit 3. { static: true } -> only for elements unconditionally in the template (never inside @if/@for) 4. QueryList is live — subscribe to .changes to react to dynamic list updates; never cache .toArray() permanently 5. Setting

template-bound properties in `ngAfterViewInit` -> use `setTimeout`, `Signals`, or `detectChanges()` to avoid `ExpressionChangedAfterItHasBeenCheckedError` 6. Prefer `@Input()/@Output()` for data flow — reach for `@ViewChild` only for imperative DOM operations (focus, scroll, animation)

The single most important Angular rule: The hook name tells you when the query is ready: `ngAfterViewInit` for `@ViewChild`; `ngAfterContentInit` for `@ContentChild` — never access either in `ngOnInit`.

Four Decorators at a Glance:

Decorator	Searches	Quantity	Available
<code>@ViewChild</code>	Own template	First match	<code>ngAfterViewInit</code>
<code>@ViewChildren</code>	Own template	All matches	<code>ngAfterViewInit</code>
<code>@ContentChild</code>	Projected content	First match	<code>ngAfterContentInit</code>
<code>@ContentChildren</code>	Projected content	All matches	<code>ngAfterContentInit</code>

`@ViewChild` vs `@Input()/@Output()` — Choose Wisely:

Use Case	Right Pattern
Passing data into a child	<code>@Input()</code>
Receiving events from a child	<code>@Output()</code> + <code>EventEmitter</code>
Focusing / scrolling a DOM element	<code>@ViewChild</code> + <code>ElementRef</code>
Calling imperative API on a child	<code>@ViewChild</code> + component instance
Querying projected child components	<code>@ContentChildren</code>
Reading child state to update parent	Refactor to <code>@Output()</code> or shared service